Dossier de projet : Klaxon v2

Raphaël Picoche

Titre professionnel Développeur Web et Web Mobile : devoir bilan

Rendu le 29 octobre 2026

Dépôt du code : github.com/Kochpi/Klaxon-V2

Sommaire

1. Introduction
2. Compétences mises en œuvre
3. Expression des besoins
4. Environnement technique
5. Maquettes et parcours utilisateur
6. Interfaces utilisateur statiques
7. Partie dynamique des interfaces
8. Base de données relationnelle
9. Accès aux données SQL
10. Accès aux données NoSQL
11. Composants métier côté serveur
12. Sécurité de l'application
13. Jeu d'essai
14. Veille sur les vulnérabilités de sécurité
15. Conclusion et bilan
16. Annexes

1. Introduction

Klaxon v2 est une application web de covoiturage entre villes : un utilisateur peut proposer un trajet, rechercher un trajet, y réserver une place, puis noter le conducteur. Je l'ai réalisée dans le cadre du devoir bilan de ma formation Développeur Web et Web Mobile.

Le projet part d'une application existante, « Touche pas au Klaxon » : un site de covoiturage entre agences d'une entreprise, écrit en PHP. Je l'ai refait avec des outils plus récents, et j'ai ajouté trois choses : une vraie liste de villes avec suggestions pendant la saisie, un

système de réservation, et des avis notés enregistrés dans une base NoSQL.

Le projet repose sur trois choix :

- **Front-end** : React, avec une interface adaptée à l'ordinateur et au téléphone.
- **Back-end** : une API Node.js / Express qui contient toute la logique métier.
- **Deux bases de données** : MySQL pour les données qui se relient entre elles (utilisateurs, villes, trajets, réservations) et MongoDB pour les avis, dont la forme est plus libre.

Le code complet est disponible sur GitHub : github.com/Kochpi/Klaxon-V2. Ce dossier présente le besoin, les choix techniques, les réalisations qui démontrent chaque compétence, puis les tests et la veille de sécurité.

2. Compétences mises en œuvre

Le projet couvre les six compétences obligatoires du titre, réparties en deux activités : le développement de la partie front-end et celui de la partie back-end.

N°	Compétence	Ce que j'ai réalisé dans Klaxon v2	Section du dossier
1	Maquetter des interfaces utilisateur web ou web mobile	Maquettes de 6 écrans web et 5 écrans mobile, avec le schéma d'enchaînement du parcours utilisateur	5
2	Réaliser des interfaces utilisateur statiques	Pages et composants React (en-tête, carte de trajet, formulaires), mise en forme en CSS adaptative	6
3	Développer la partie dynamique des interfaces	Connexion, recherche avec autocomplétion, réservation et avis : état React et appels à l'API	7
4	Mettre en place une base de données relationnelle	Base MySQL de 4 tables avec clés étrangères et contraintes d'intégrité, script de création et jeu de données	8
5	Développer des composants d'accès aux données SQL et NoSQL	Requêtes SQL préparées (MySQL) et collection d'avis avec agrégation pour la moyenne (MongoDB)	9 et 10

N°	Compétence	Ce que j'ai réalisé dans Klaxon v2	Section du dossier
6	Développer des composants métier côté serveur	Règles de réservation, gestion des places, droits sur les trajets, avis réservés aux passagers, calcul de la note moyenne	11

Deux autres compétences du titre sont aussi mobilisées, sans faire l'objet d'une section dédiée : installer et configurer son environnement de travail (section 4), et documenter le déploiement (le projet tourne en local pour le moment, voir la conclusion).

La sécurité est un fil rouge : elle est présentée à part en section 12, puis validée par un jeu d'essai (section 13) et une veille de vulnérabilités (section 14).

3. Expression des besoins

3.1 Contexte et objectif

Se déplacer d'une ville à une autre coûte cher quand on est seul dans la voiture. Klaxon v2 met en relation des conducteurs qui ont des places libres et des passagers qui cherchent un trajet, pour partager les frais et limiter le nombre de voitures sur la route. L'objectif du projet est de livrer une application complète, utilisable sur ordinateur comme sur téléphone, dont les données et les droits d'accès sont protégés.

3.2 Public cible

L'application s'adresse à toute personne qui voyage entre villes de France : étudiants, salariés, particuliers. Il n'y a qu'un seul type de compte : le même utilisateur peut proposer un trajet comme conducteur, et réserver celui d'un autre comme passager.

3.3 Besoins fonctionnels

Besoin	Description	Accès
Créer un compte, se connecter, se déconnecter	Inscription avec nom, prénom, téléphone, email et mot de passe	Public
Consulter les trajets	Liste des trajets à venir avec le nombre de places restantes	Public
Rechercher un trajet	Filtres par ville de départ, ville d'arrivée et date, avec autocomplétion des villes	Public

Besoin	Description	Accès
Voir le détail d'un trajet	Horaires, places, conducteur, note moyenne, avis ; email et téléphone du conducteur visibles uniquement une fois connecté	Connecté
Proposer un trajet	Villes, dates et heures, nombre de places (1 à 9)	Connecté
Supprimer son trajet	Uniquement un trajet dont on est le conducteur	Connecté
Réserver une place	Une réservation = une place ; impossible sur son propre trajet, un trajet complet ou déjà parti	Connecté
Annuler sa réservation	La place est rendue au trajet	Connecté
Donner un avis	Note de 1 à 5 et commentaire, réservé aux passagers du trajet, un seul avis par passager et par trajet	Connecté (passager)
Consulter la note moyenne d'un conducteur	Moyenne arrondie à une décimale et nombre d'avis	Public

3.4 Besoins non fonctionnels

- **Sécurité** : mots de passe protégés, accès limité aux données de l'utilisateur, protection contre les attaques courantes (injection, XSS, force brute).
- **Ergonomie** : interface lisible et utilisable à une main sur téléphone, parcours court de la recherche à la réservation.
- **Intégrité des données** : les règles importantes (dates cohérentes, nombre de places, pas de double réservation) sont vérifiées par le serveur et par la base.

3.5 Limites du projet

Pour rester réalisable dans le temps imparti, certains points sont volontairement hors périmètre :

- un seul rôle d'utilisateur (un rôle administrateur est une piste d'évolution) ;
- pas de modification d'un trajet : on le supprime et on le recrée ;
- pas de paiement ni de messagerie entre utilisateurs ;

- pas de transaction SQL pour la dernière place d'un trajet : si deux personnes la réservent exactement en même temps, les deux pourraient être acceptées (limite connue, voir section 11.2) ;
- application hébergée en local pour le moment.

4. Environnement technique

4.1 Langages, frameworks et bibliothèques

Partie	Technologie	Rôle dans le projet
Front-end	React 19	Construction de l'interface en composants
Front-end	Vite 8	Serveur de développement et outil de compilation
Front-end	react-router-dom 7	Navigation entre les pages sans rechargement
Front-end	CSS	Mise en forme adaptative (ordinateur et téléphone)
Back-end	Node.js et Express 5	API qui reçoit les requêtes et applique les règles métier
Base relationnelle	MySQL / MariaDB (XAMPP), pilote mysql2	Utilisateurs, villes, trajets, réservations
Base NoSQL	MongoDB Atlas, pilote mongodb 7	Avis et notes
Sécurité	bcrypt	Hachage des mots de passe
Sécurité	jsonwebtoken et cookie-parser	Session par jeton, stocké dans un cookie
Sécurité	helmet, cors, express-rate-limit	En-têtes HTTP sécurisés, origine autorisée, limitation des tentatives de connexion
Configuration	dotenv	Secrets et paramètres dans un fichier `.env` hors du dépôt
Outil de développement	nodemon	Redémarrage automatique de l'API à chaque modification

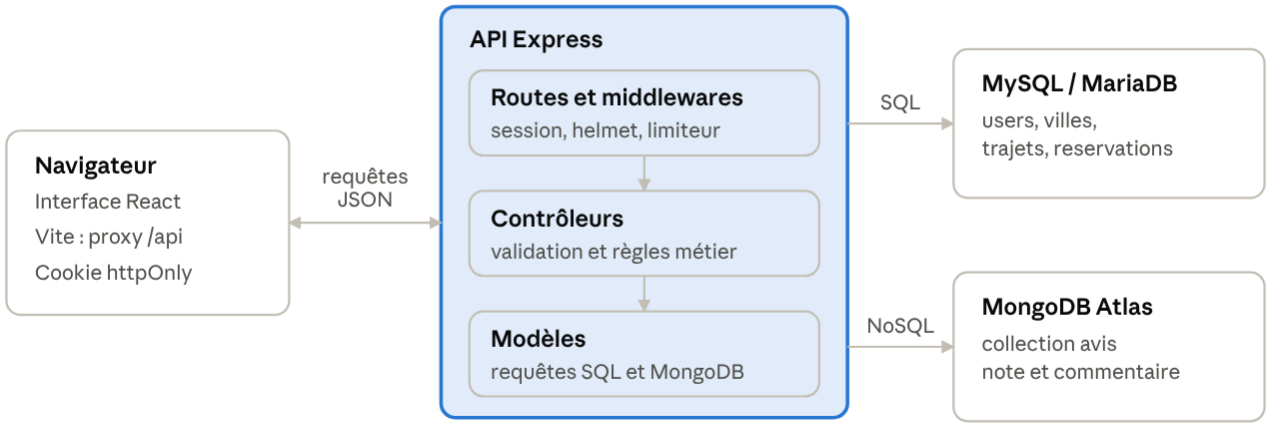
4.2 Outils

- **Visual Studio Code** pour écrire le code.
- **Git et GitHub** pour versionner : une branche par fonctionnalité, fusionnée dans `main` par une Pull Request.
- **Postman** pour tester l'API sans passer par l'interface.
- **phpMyAdmin** pour créer et consulter la base MySQL, et **MongoDB Compass** pour consulter les avis.
- **Chrome et ses outils de développement** pour tester le rendu, le mode mobile et les cookies, et **Figma** pour dessiner les maquettes.

4.3 Architecture générale

L'application suit une architecture en trois parties. Le navigateur affiche l'interface React. Quand l'utilisateur agit, React envoie une requête à l'API Express. L'API vérifie la session, applique les règles métier, puis lit ou écrit dans la bonne base : MySQL pour les données liées entre elles, MongoDB pour les avis.

Seule l'API accède aux deux bases de données



architecture générale · navigateur, API, deux bases

Le navigateur ne parle jamais directement aux bases de données : tout passe par l'API, qui vérifie chaque demande avant de lire ou d'écrire.

5. Maquettes et parcours utilisateur (compétence 1)

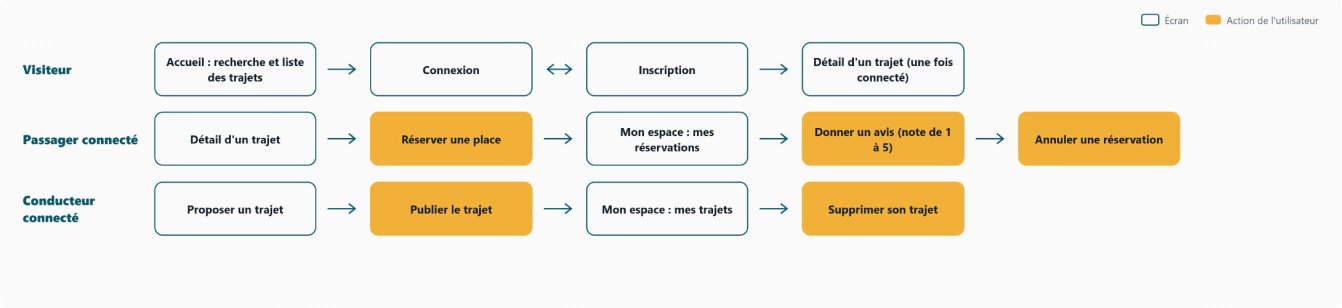
Avant d'écrire le code du front-end, j'ai dessiné sous Figma les maquettes de 6 écrans pour l'ordinateur et de 5 écrans pour le téléphone, ainsi que le schéma d'enchaînement entre ces écrans. Elles m'ont servi de plan pour la mise en page et pour découper l'interface en composants.

5.1 Charte graphique

Élément	Choix	Usage
Bleu-vert `#0E5A6B`	Couleur principale	Logo, titres, liens, boutons principaux
Jaune `#F5B53D`	Couleur d'action	Boutons qui déclenchent une action (Rechercher, Réserver, Publier)
Gris clair `#F4F6F8`	Fond de page	Fait ressortir les cartes blanches
Rouge `#B3261E`	Alerte	Suppression, annulation, messages d'erreur
Vert `#14633F`	Confirmation	Places disponibles, messages de succès
Police du système	Typographie	Texte lisible sur tous les appareils, sans chargement externe

Tout est organisé en cartes aux coins arrondis, et le jaune est réservé aux boutons d'action pour que l'action principale se repère immédiatement.

5.2 Schéma d'enchaînement des écrans

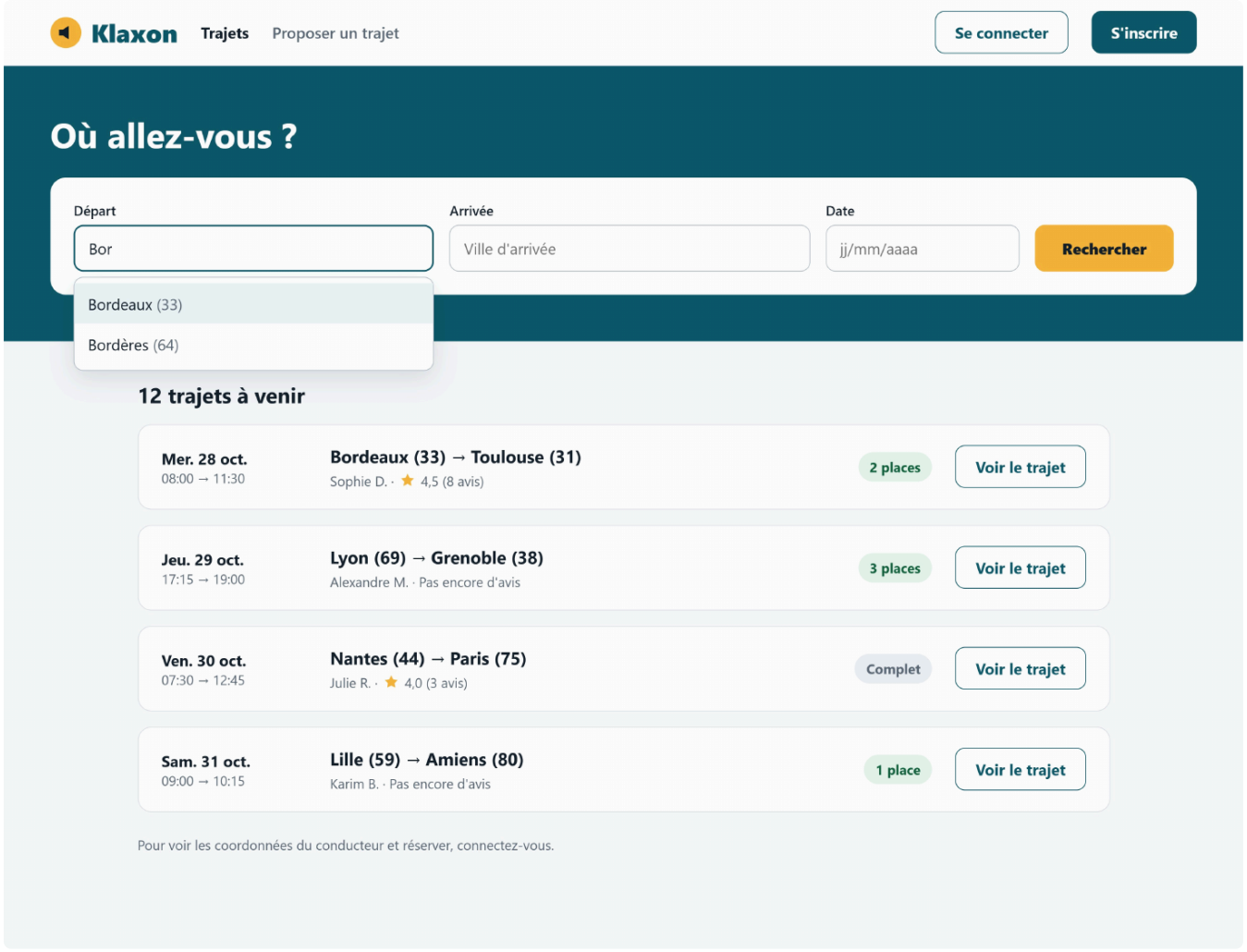


Parcours utilisateur : les écrans sont encadrés, les actions de l'utilisateur sont en jaune.

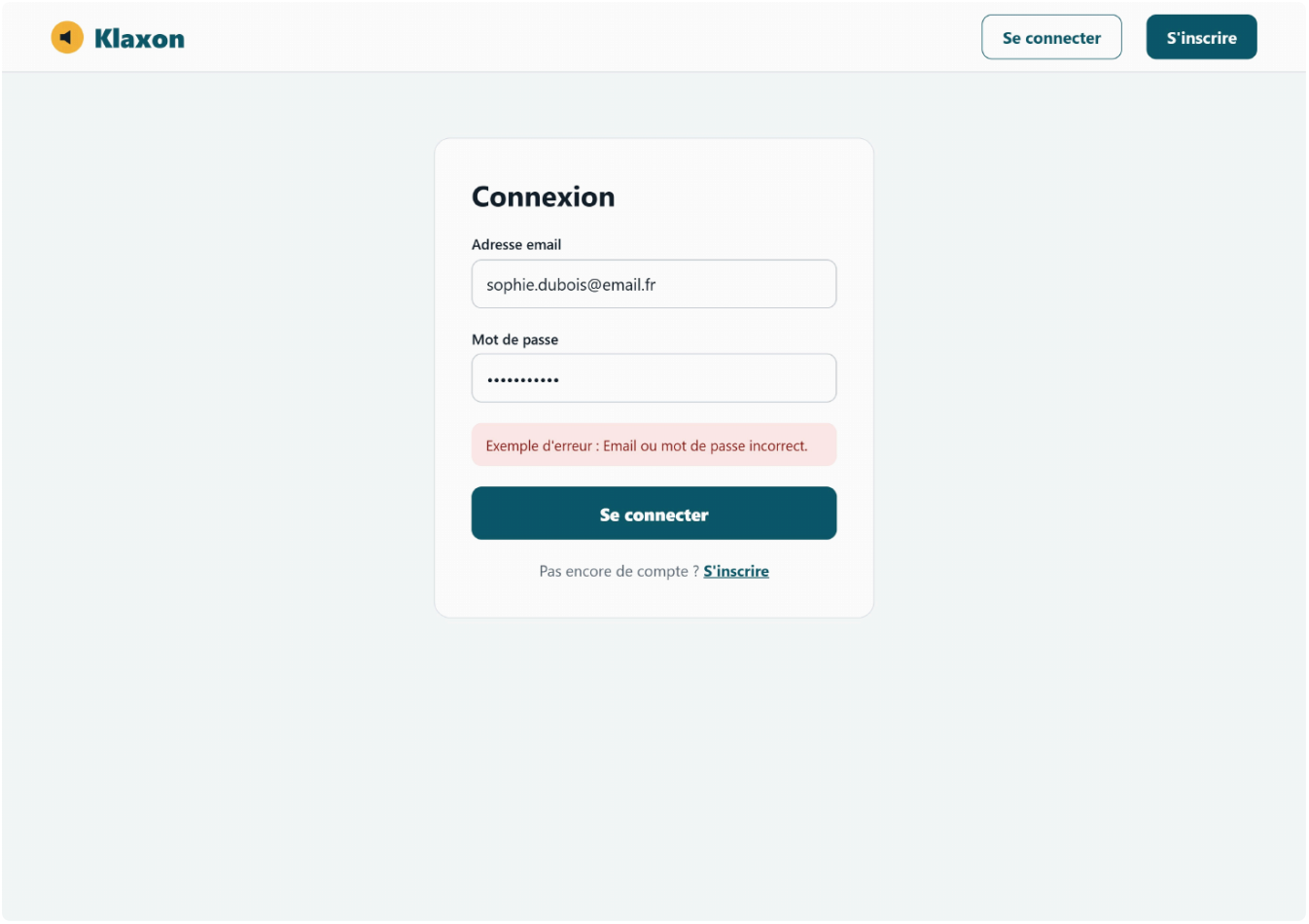
Trois parcours sont prévus :

- **Visiteur** : il consulte la liste des trajets sur l'accueil, puis se connecte ou s'inscrit. Le détail d'un trajet n'est accessible qu'une fois connecté.
- **Passager connecté** : depuis le détail d'un trajet, il réserve une place, la retrouve dans Mon espace, peut ensuite donner un avis ou annuler sa réservation.
- **Conducteur connecté** : il propose un trajet, le retrouve dans Mon espace et peut le supprimer.

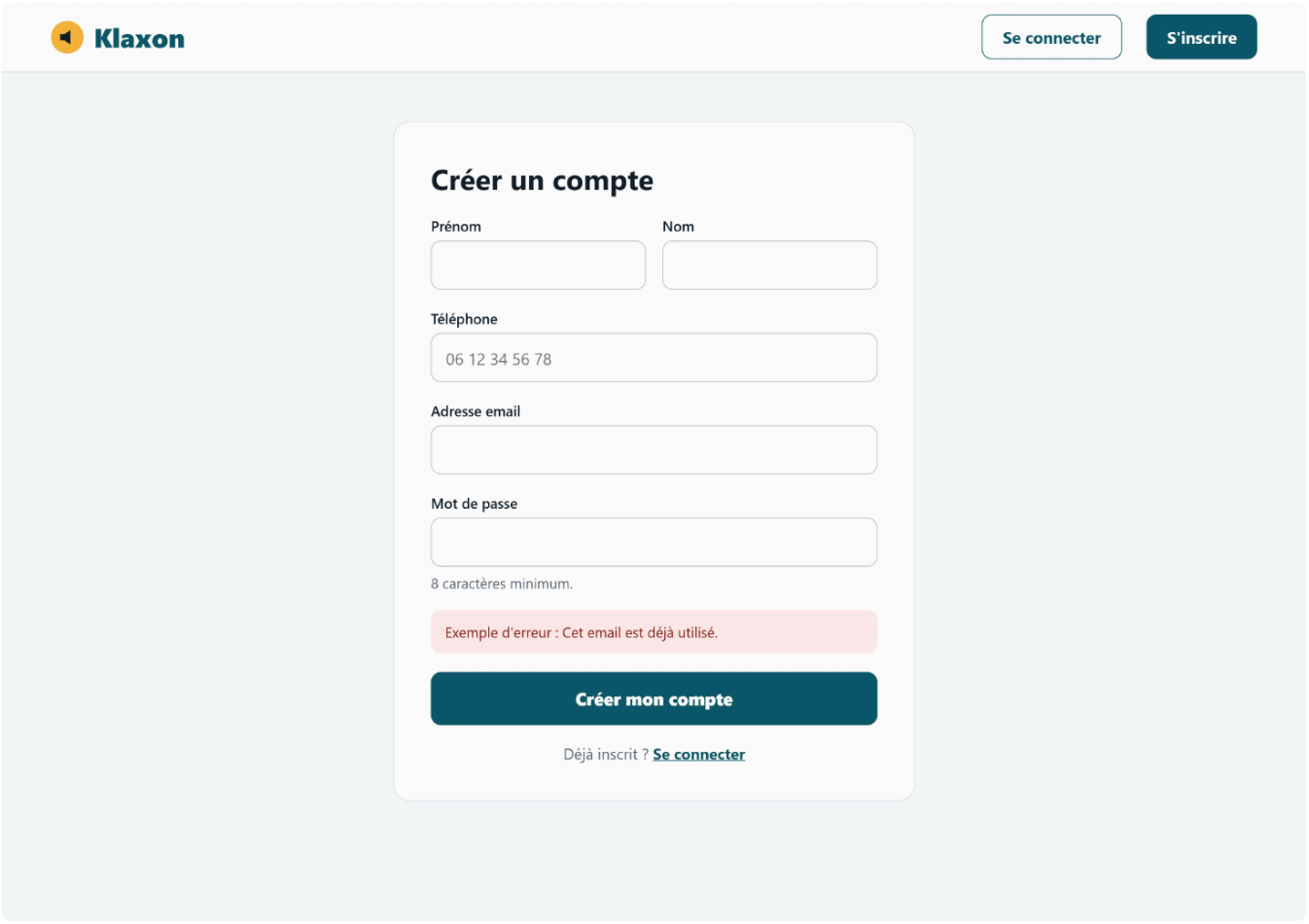
5.3 Maquettes de la version ordinateur



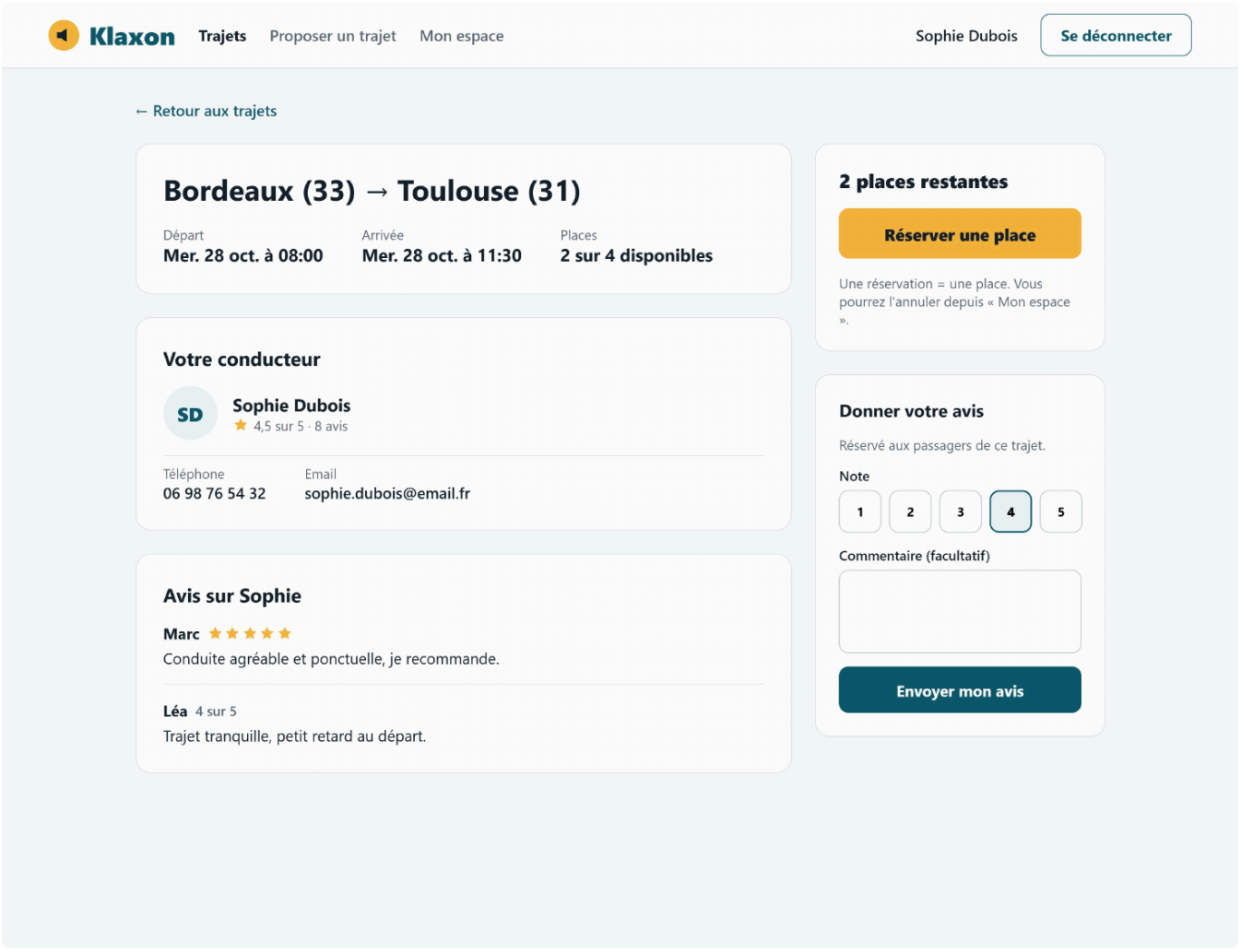
Accueil : un formulaire de recherche (départ, arrivée, date) au-dessus de la liste des trajets à venir. Les trajets complets sont signalés en gris.



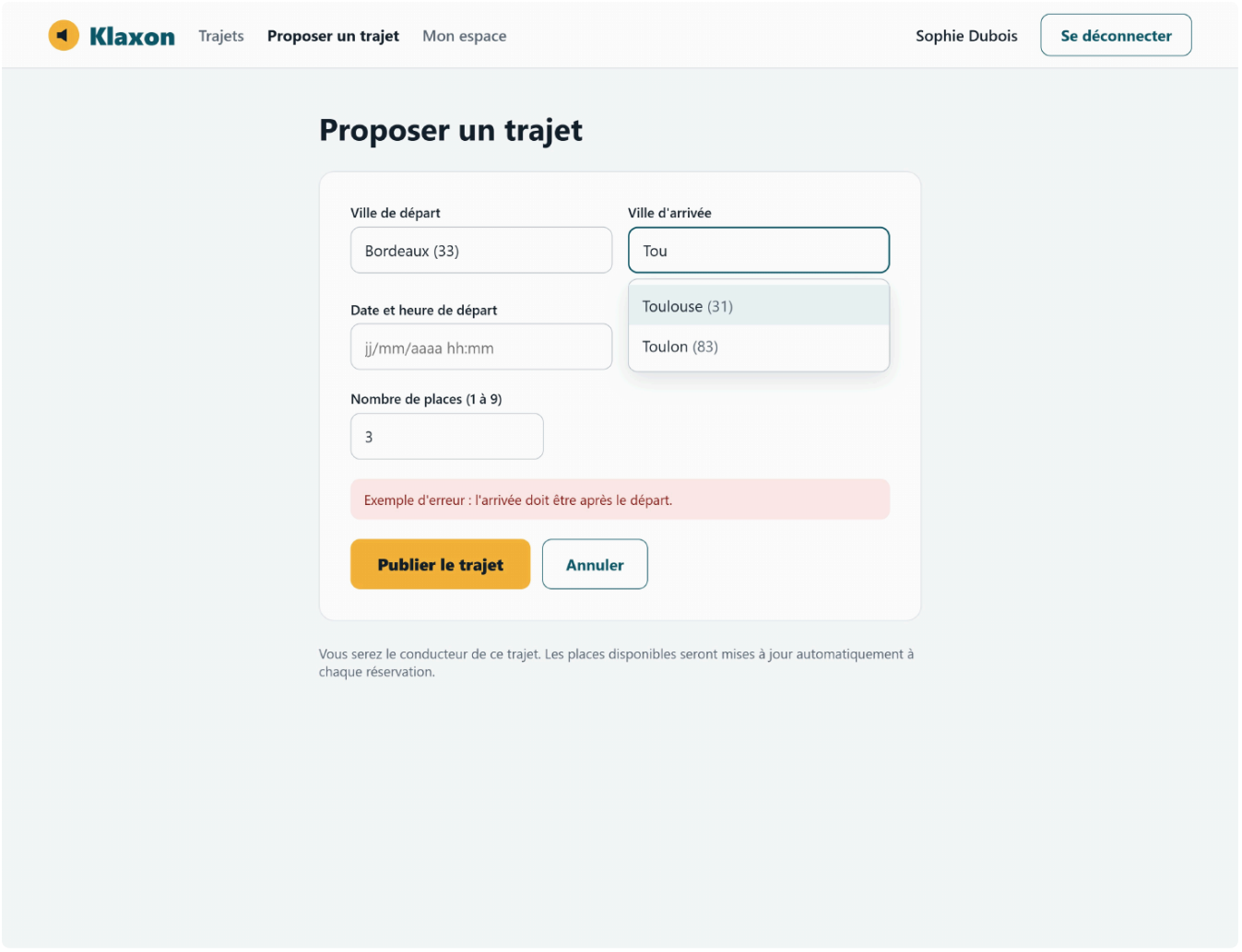
Connexion : un formulaire court, centré dans une carte, avec un lien vers l'inscription.



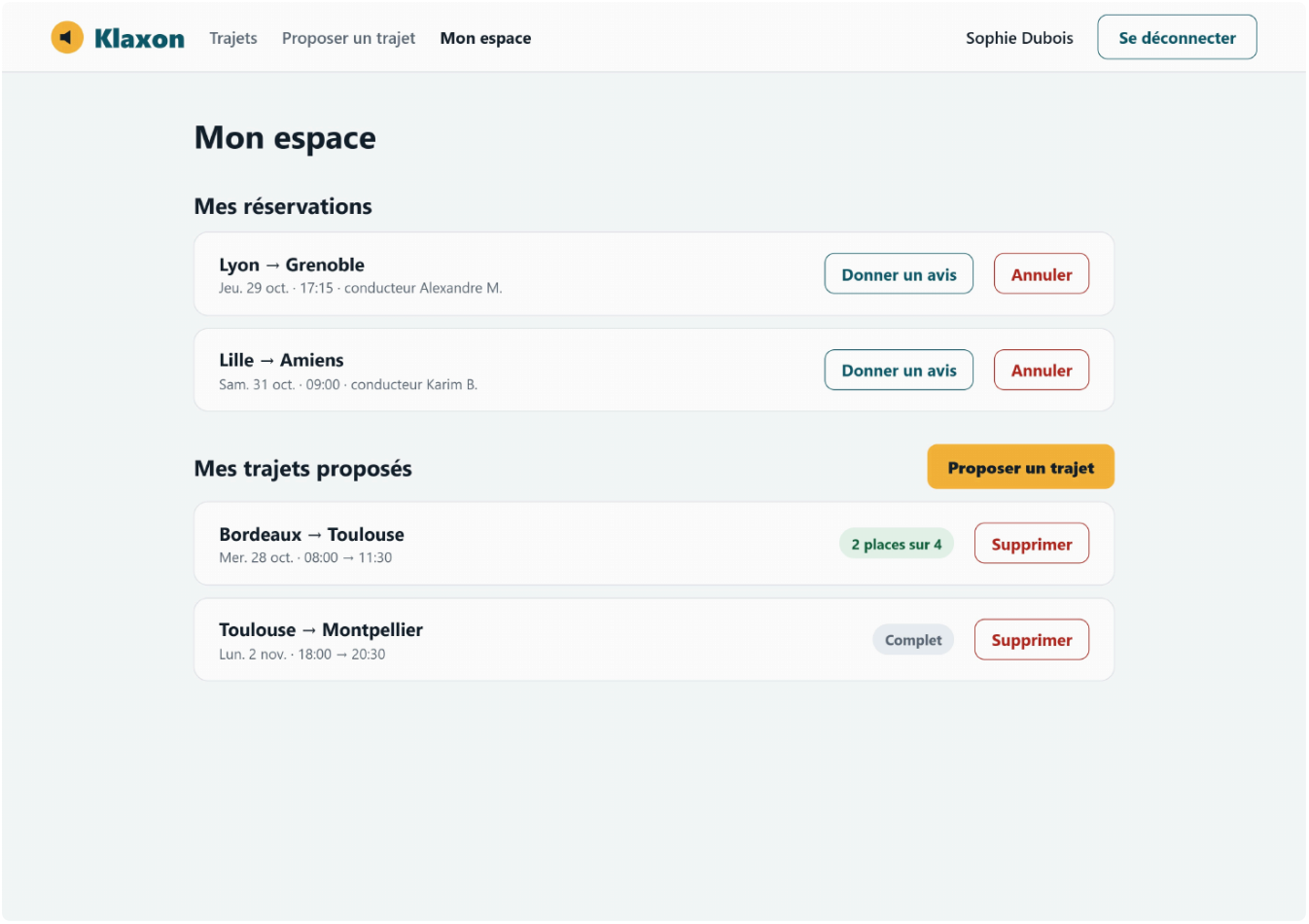
Inscription : cinq champs (prénom, nom, téléphone, email, mot de passe) et un rappel de la longueur minimale du mot de passe.



Détail d'un trajet : horaires et places, fiche du conducteur avec sa note moyenne et ses contacts, bouton de réservation et avis des passagers.

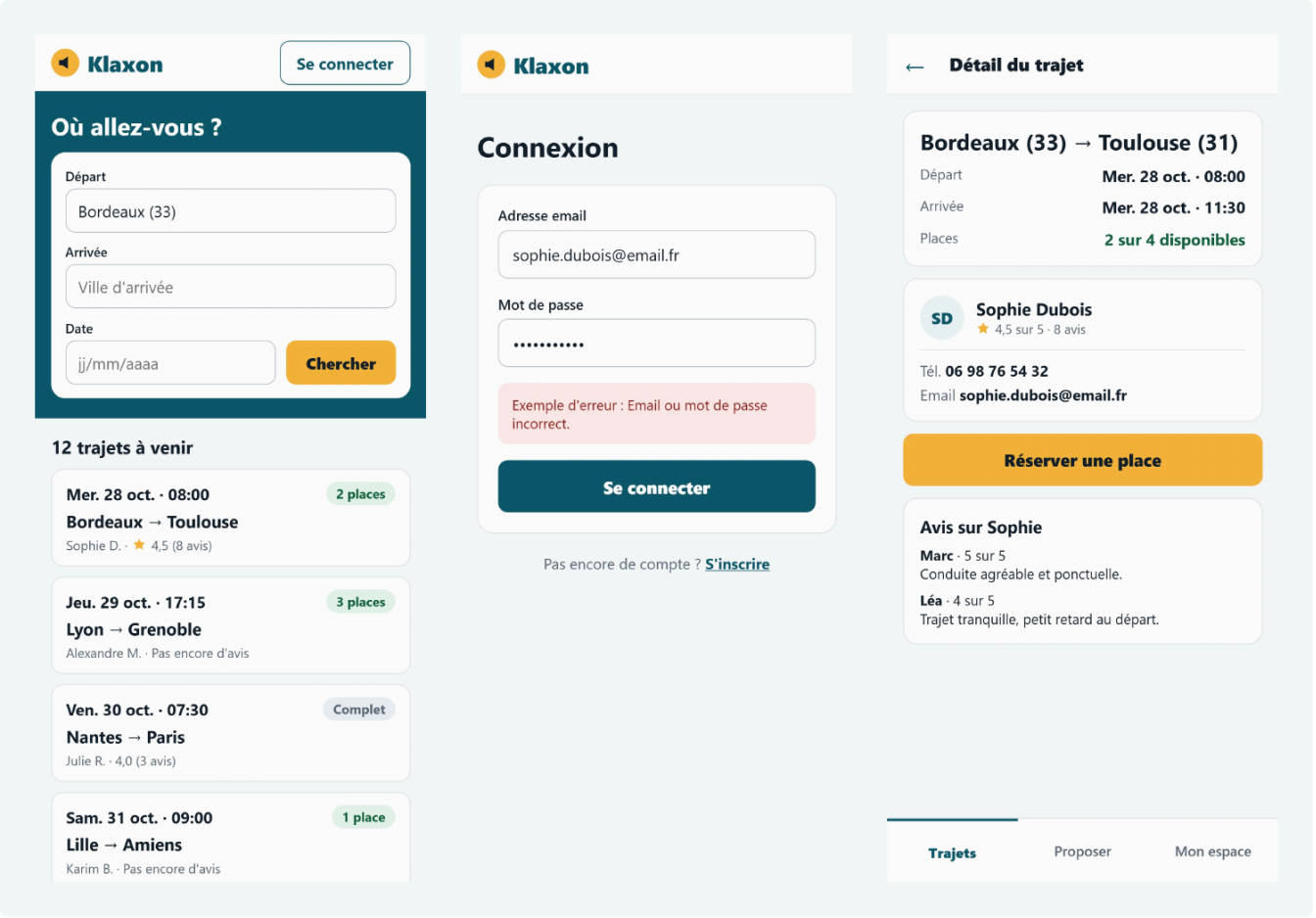


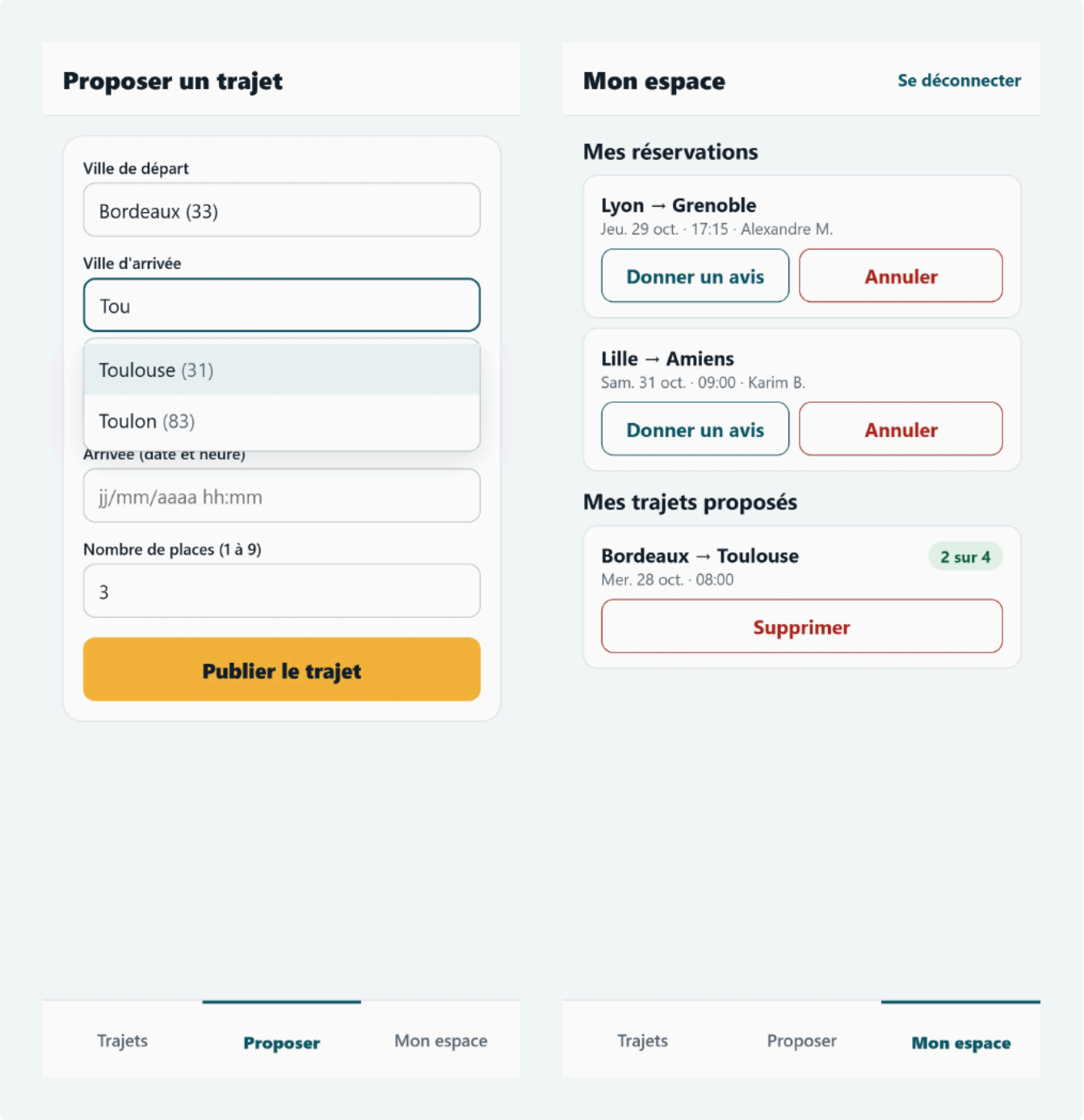
Proposer un trajet : villes avec autocomplétion, dates et heures, nombre de places.



Mon espace : les réservations de l'utilisateur (avec les actions Donner un avis et Annuler) et les trajets qu'il propose (avec l'action Supprimer).

5.4 Maquettes de la version téléphone





Maquettes mobile de 390 × 844 pixels : accueil, connexion, détail d'un trajet, proposer un trajet et mon espace.

5.5 Adaptation au mobile

La version téléphone n'est pas la version ordinateur rétrécie : l'organisation change pour tenir compte de l'usage à une main.

- Une **barre de navigation fixe en bas** (Trajets, Proposer, Mon espace) remplace le menu du haut, plus facile à atteindre avec le pouce.
- Les blocs s'empilent verticalement : les champs de recherche passent l'un sous l'autre, et la fiche du trajet laisse la place à la réservation.

- Les boutons et champs font au moins 44 pixels de haut, la taille minimale recommandée pour un doigt.
- Dans Mon espace, chaque réservation devient une carte avec ses deux boutons côte à côte.

6. Interfaces utilisateur statiques (compétence 2)

L'interface est construite avec React. Chaque écran est une page, et chaque morceau réutilisable d'un écran (le menu, une ligne de trajet…) est un composant. Cette section montre la partie « statique » : ce qui est affiché et comment c'est mis en forme. Ce qui réagit aux actions de l'utilisateur est en section 7.

6.1 Pages et composants

Élément	Fichier	Rôle
Page	`Accueil.jsx`	Recherche et liste des trajets
Page	`Detail.jsx`	Détail d'un trajet, réservation, avis
Page	`Connexion.jsx`, `Inscription.jsx`	Formulaires pour accéder au compte
Page	`Proposer.jsx`	Formulaire pour créer un trajet
Page	`MonEspace.jsx`	Réservations et trajets de l'utilisateur
Composant	`Header.jsx`	Menu du haut, qui change si l'on est connecté
Composant	`TrajetCard.jsx`	Une ligne de la liste des trajets
Composant	`VilleInput.jsx`	Champ de ville avec suggestions
Composant	`AvisForm.jsx`	Formulaire de note et de commentaire

Le site change de page sans se recharger, grâce à `react-router-dom`. Chaque adresse est reliée à une page, et les pages privées sont entourées d'un contrôle de connexion (section 7.3).

6.2 Exemple : la ligne d'un trajet

Le composant `TrajetCard` reçoit un trajet et affiche ses informations. Sa seule « règle » : il affiche « Complet » quand il n'y a plus de place, et le nombre de places sinon.

```
export default function TrajetCard({ trajet }) {
  const complet = trajet.places_dispo === 0

  return (
    <article className="card trajet">
      <strong>{formatDate(trajet.date_depart)}</strong>
      <div>{trajet.ville_depart} → {trajet.ville_arrivee}</div>
      <span className="badge">
        {complet ? 'Complet' : `${trajet.places_dispo} place(s)`}
      </span>
      <Link to={`/trajets/${trajet.id}`}>Voir le trajet</Link>
    </article>
  )
}
```

(Extrait raccourci : le fichier complet est dans le dépôt.)

6.3 Mise en forme

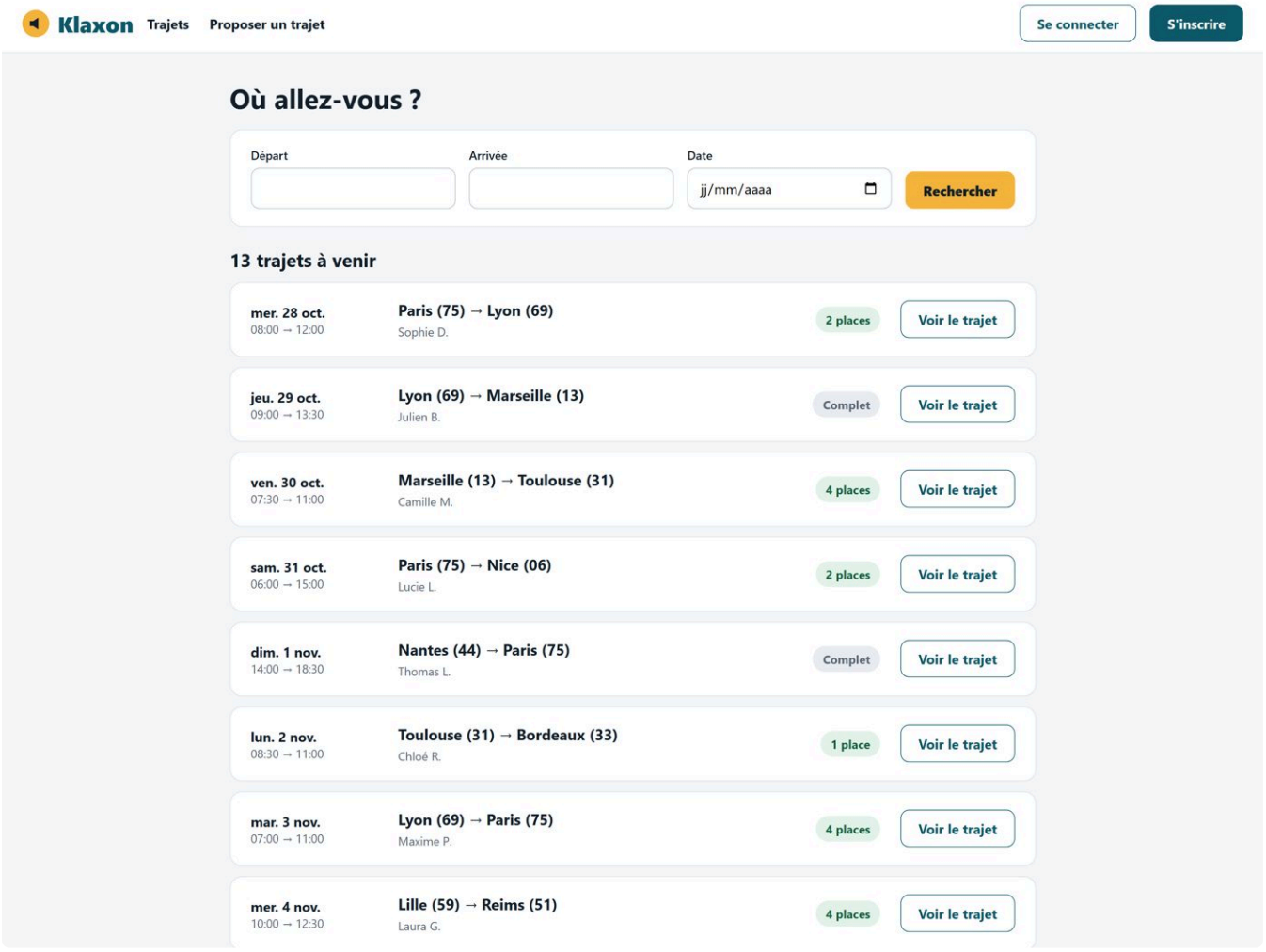
Le style est écrit en CSS. Les couleurs de la charte (section 5.1) sont déclarées une seule fois, puis réutilisées partout : pour changer une couleur, on la modifie à un seul endroit.

```
:root {
  --principal: #0e5a6b;
  --accent: #f5b53d;
  --fond: #f4f6f8;
}

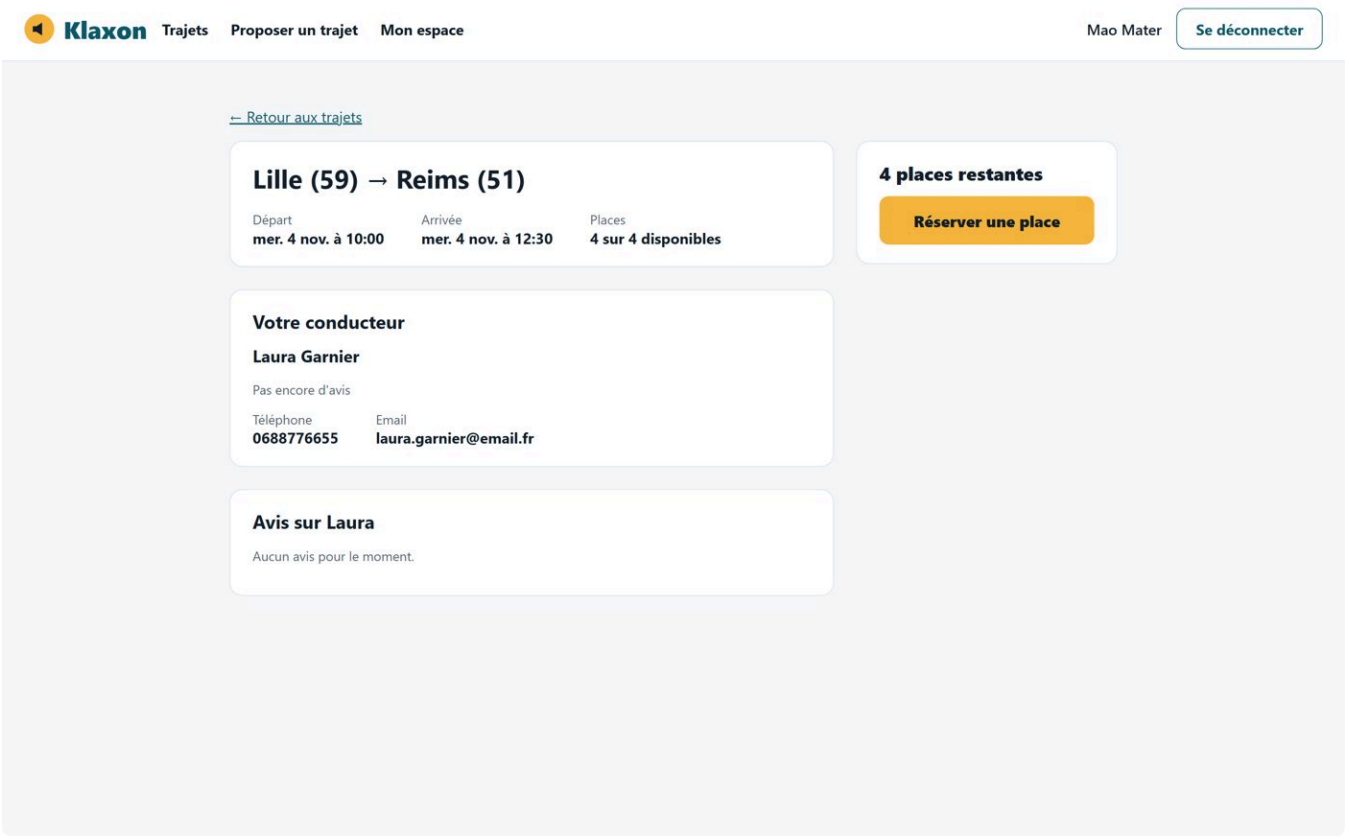
.card {
  background: #fff;
  border-radius: 14px;
  padding: 20px 24px;
}
```

Sur téléphone, une règle CSS (`@media`) s'applique quand l'écran fait moins de 600 pixels de large : les marges et les titres sont réduits. Sur la page de détail, les deux colonnes de l'ordinateur passent l'une sous l'autre. Les boutons font au moins 44 pixels de haut pour être faciles à toucher.

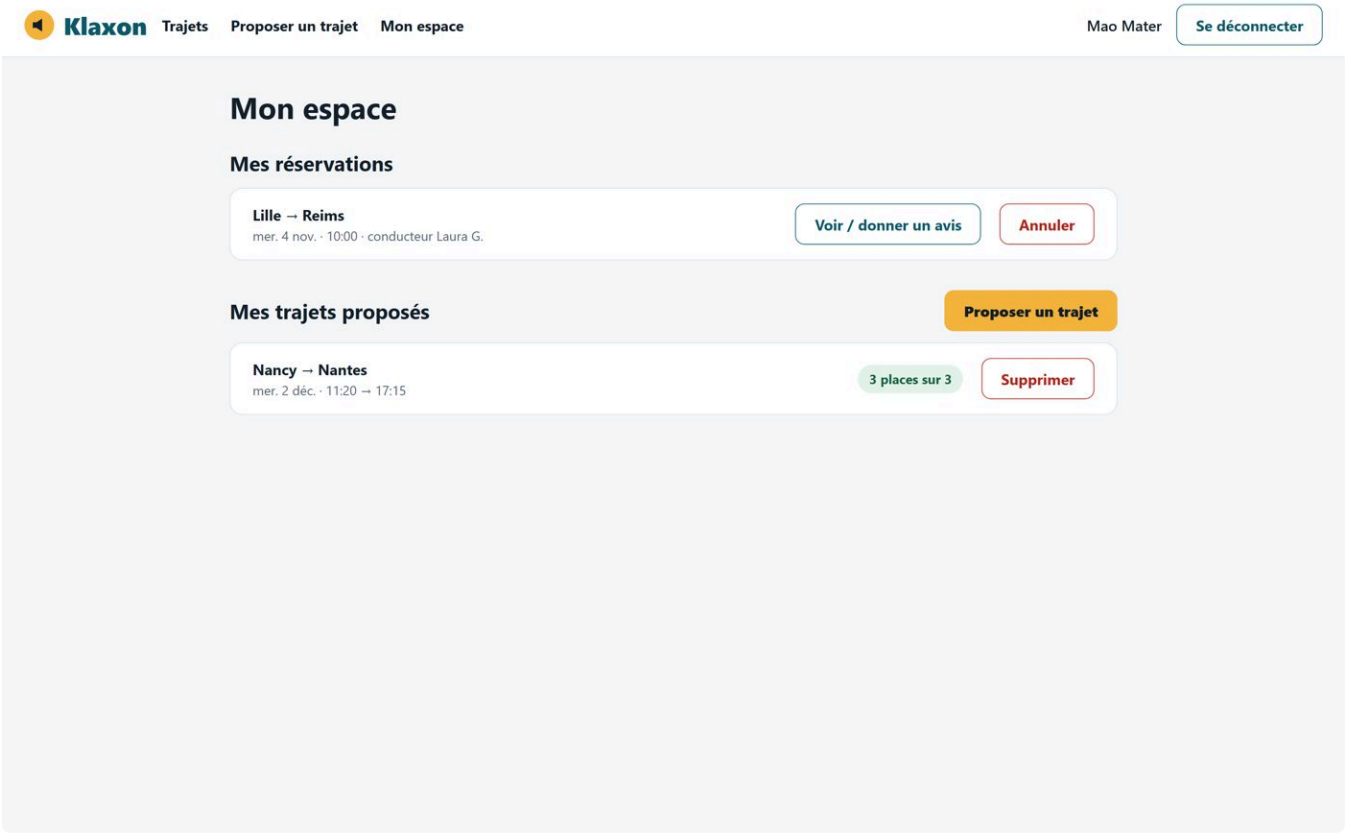
6.4 Résultat : version ordinateur



Accueil : formulaire de recherche et liste des trajets à venir.

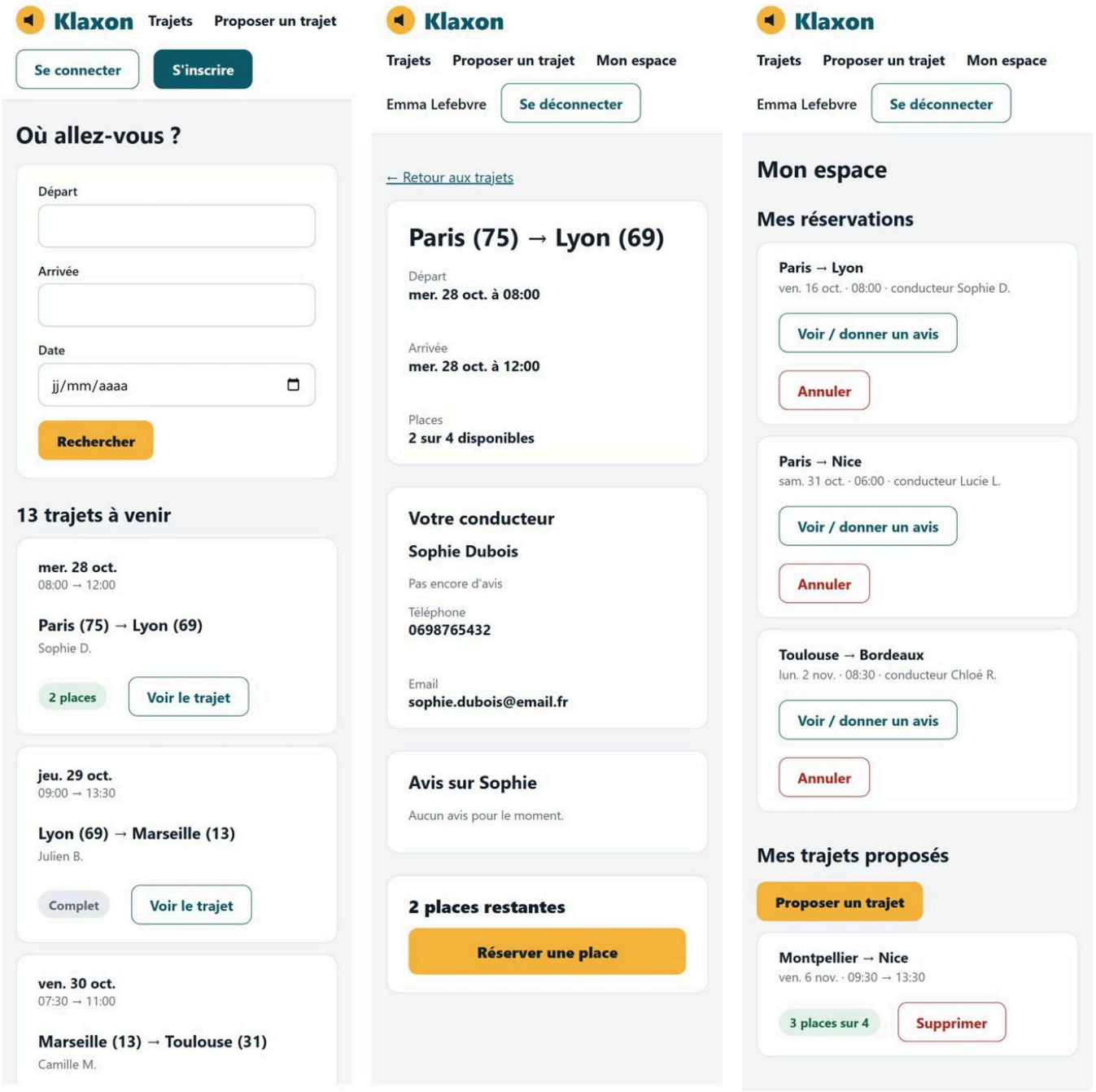


Détail d'un trajet : deux colonnes, avec la réservation à droite.



Mon espace : réservations et trajets proposés.

6.5 Résultat : version téléphone



Accueil, détail d'un trajet et Mon espace sur téléphone : les blocs sont empilés.

6.6 Différence avec les maquettes

Le résultat suit les maquettes, sauf sur un point : sur téléphone, le menu reste en haut de l'écran au lieu d'être une barre fixe en bas. Je l'ai gardé pour que le CSS reste simple ; c'est une amélioration possible.

7. Partie dynamique des interfaces (compétence 3)

La partie dynamique, c'est tout ce qui réagit aux actions de l'utilisateur : se connecter, chercher un trajet, réserver, donner un avis. Pour cela, l'interface doit retenir des informations (ce que l'utilisateur a tapé, la liste des trajets) et aller en chercher auprès de l'API. React m'offre trois outils simples pour ça.

Outil React	À quoi il sert dans Klaxon v2
`useState`	Garder une information en mémoire : les trajets affichés, le texte d'un champ, un message d'erreur. Quand elle change, l'écran se met à jour tout seul.
`useEffect`	Lancer une action quand la page s'affiche, par exemple demander la liste des trajets à l'API.
Un contexte	Partager une information entre toutes les pages, ici : qui est connecté.

7.1 Parler à l'API

Toutes les demandes au serveur passent par un seul petit fichier, `api.js`. Il envoie la demande, lit la réponse, et si le serveur répond par une erreur, il en fait un message que l'écran peut afficher (par exemple « Email ou mot de passe incorrect »). Les pages n'ont donc jamais à gérer ces détails.

```
const response = await fetch('/api' + url, options)
const data = await response.json()
if (!response.ok) throw new Error(data.message || 'Une erreur est survenue')
```

7.2 Afficher et rechercher les trajets

Quand la page d'accueil s'ouvre, elle demande la liste des trajets à l'API et la range en mémoire. Pendant l'attente, elle affiche « Chargement… ». Quand l'utilisateur clique sur Rechercher, la page redemande la liste en ajoutant les filtres choisis (départ, arrivée, date).

```
useEffect(() => {
  api.get('/trajets').then((data) => setTrajets(data.trajets))
}, [])
```

Les champs de ville fonctionnent sur le même principe : à chaque lettre tapée, l'application demande les villes qui correspondent et propose les six premières.

7.3 Savoir qui est connecté

Le jeton de connexion est caché dans un cookie que le code de la page ne peut pas lire (section 12). Au démarrage, l'application demande donc à l'API « qui suis-je ? » et met la réponse à disposition de toutes les pages. Les pages réservées (détail, proposer, mon espace) sont entourées d'un gardien : si personne n'est connecté, il renvoie vers la page de connexion, puis ramène l'utilisateur là où il voulait aller.

```
if (!user) return <Navigate to="/connexion" state={{ from: location.pathname
}} replace />
```

Ce gardien ne sert qu'au confort d'affichage. La vraie protection est côté serveur, qui vérifie le cookie à chaque demande.

7.4 Réserver une place

Sur le détail d'un trajet, le bouton « Réserver une place » envoie la demande à l'API. Si elle est acceptée, la page se recharge et affiche « Réservation confirmée ». Si elle est refusée, le message du serveur s'affiche (trajet complet, déjà réservé…). Les boutons s'adaptent : pas de bouton sur son propre trajet, bouton grisé si le trajet est complet.

```
await api.post('/reservations', { trajet_id: trajet.id })
recharger()
```

7.5 Donner un avis

Le formulaire d'avis garde en mémoire la note (de 1 à 5) et le commentaire. À l'envoi, il les transmet à l'API. Si le serveur refuse (avis déjà donné, utilisateur qui n'est pas passager), l'erreur s'affiche dans le formulaire. Sinon, la page recharge la liste des avis et la note moyenne. Le code complet de cette fonctionnalité est en annexe.

```
await api.post('/avis', { trajet_id: trajetId, note, commentaire })
onDone() // la page recharge les avis
```

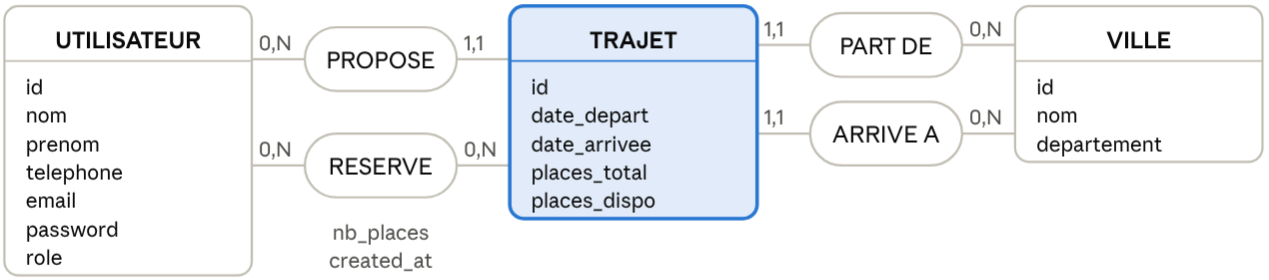
8. Base de données relationnelle (compétence 4)

Les données qui se relient entre elles (utilisateurs, villes, trajets, réservations) sont dans une base MySQL. Je l'ai conçue en deux temps : d'abord un schéma conceptuel, qui dit ce qu'on stocke et comment les éléments sont liés, puis un schéma physique, qui est la version réellement créée dans MySQL.

8.1 Schéma conceptuel

Le schéma conceptuel (MCD) montre les trois éléments de base et leurs liens. Les nombres près des liens disent combien de fois chaque élément peut intervenir : « 1,1 » veut dire exactement un, « 0,N » veut dire zéro ou plusieurs.

Un trajet : un conducteur, deux villes, des passagers



Un trajet a un seul conducteur et deux villes (départ et arrivée).
La réservation relie un passager à un trajet : elle porte le nombre de places et la date.

schéma conceptuel (MCD) · 3 entités, 4 associations

Par exemple, un trajet a **exactement un** conducteur, mais un utilisateur peut en proposer **plusieurs**. Un utilisateur peut aussi réserver plusieurs trajets, et un trajet peut être réservé par plusieurs utilisateurs : ce lien « plusieurs à plusieurs » devient la table `reservations`.

8.2 Schéma physique

Le schéma physique contient quatre tables. Voici la capture du concepteur de phpMyAdmin : les traits relient chaque clé étrangère à la table qu'elle désigne.

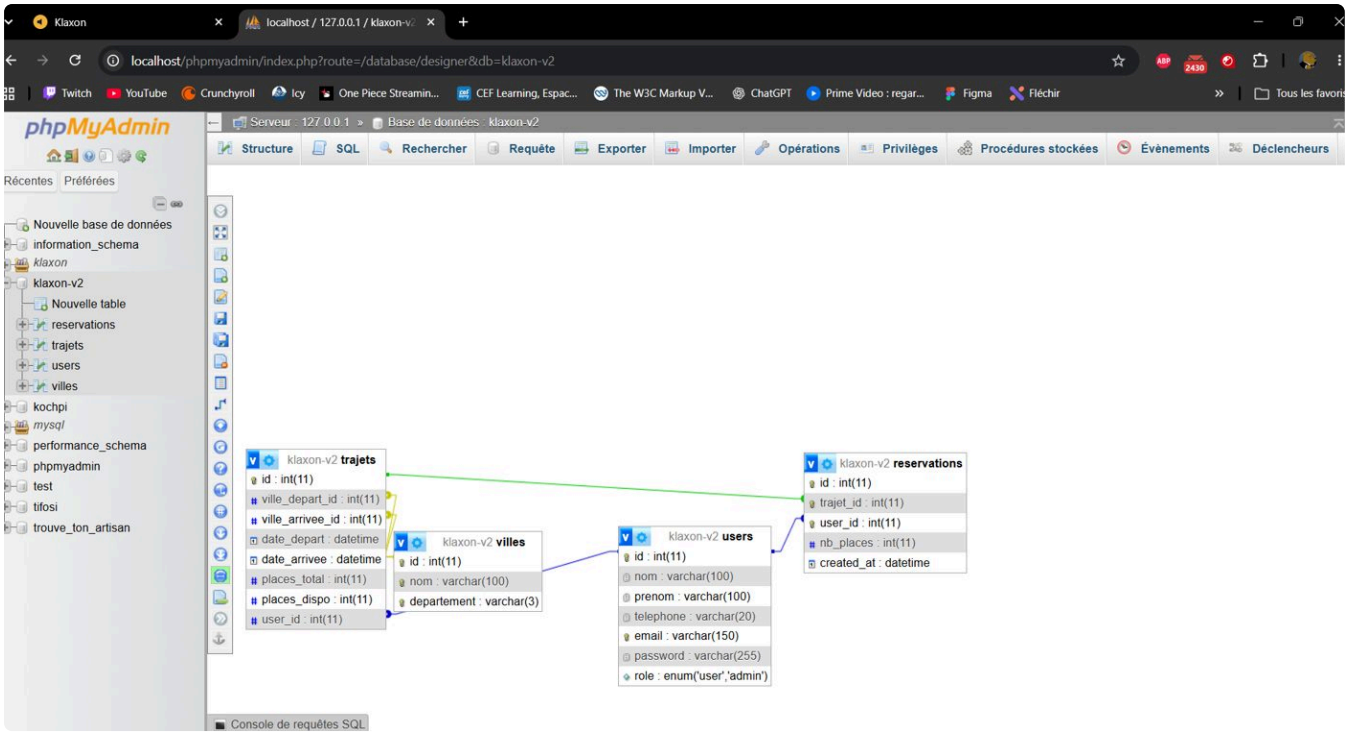


Schéma physique : les tables `users`, `villes`, `trajets` et `reservations`.

Table	Contenu	Liens
`users`	Les comptes : nom, prénom, téléphone, email, mot de passe haché	Aucun
`villes`	Nom et département de chaque ville	Aucun
`trajets`	Horaires, places, villes de départ et d'arrivée, conducteur	Vers `villes` (deux fois) et `users`
`reservations`	Qui a réservé quel trajet, combien de places	Vers `trajets` et `users`

Le département est stocké avec la ville : il distingue les villes qui portent le même nom et permet d'afficher « Lyon (69) » dans les suggestions.

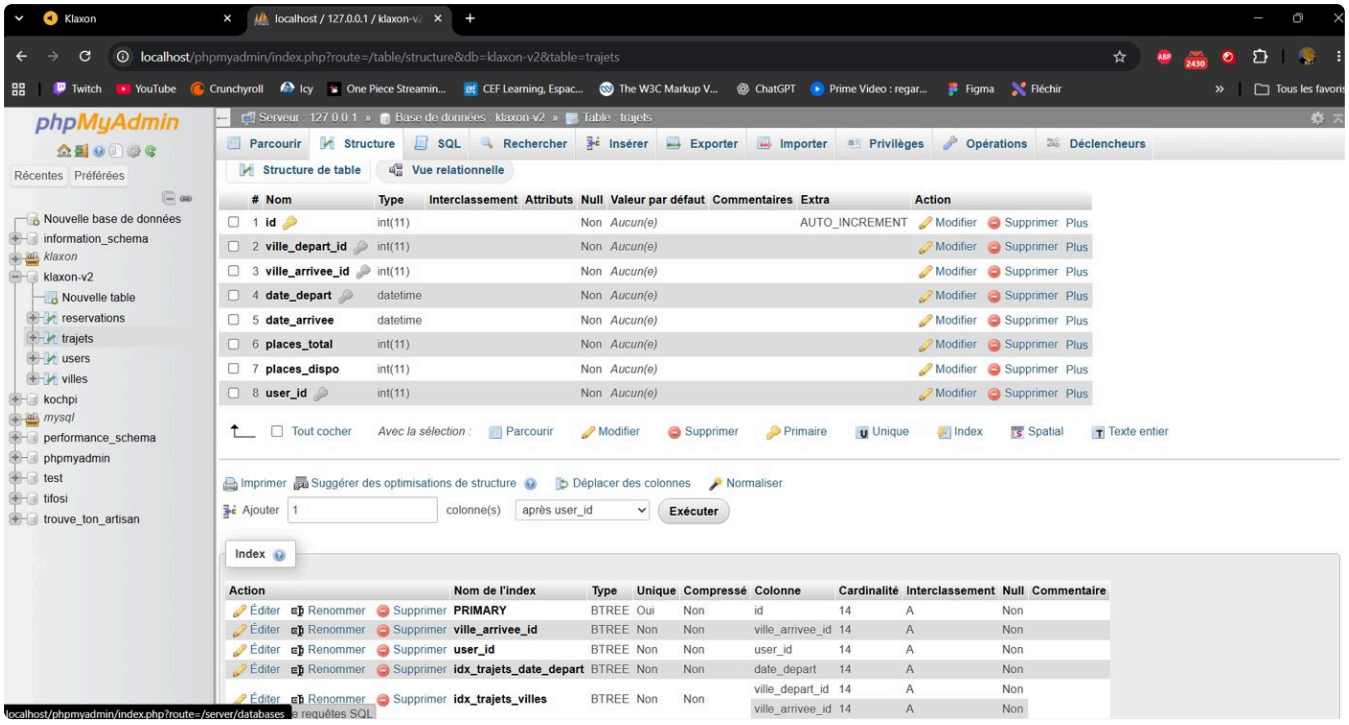
8.3 Règles d'intégrité

La base elle-même refuse les données incohérentes, même si le serveur avait un oubli. C'est une deuxième protection derrière celle du code.

Règle	Comment elle est écrite
Un email ne sert qu'une fois	`UNIQUE` sur `users.email`
Une réservation par passager et par trajet	`UNIQUE (user_id, trajet_id)`

Règle	Comment elle est écrite
Un trajet ne peut pas relier une ville à elle-même	`CHECK (ville_depart_id <> ville_arrivee_id)`
L'arrivée est après le départ	`CHECK (date_arrivee > date_depart)`
Entre 1 et 9 places	`CHECK (places_total BETWEEN 1 AND 9)`
Jamais de places négatives ni en trop	`CHECK (places_dispo >= 0 AND places_dispo <= places_total)`
Un trajet supprimé emporte ses réservations	`ON DELETE CASCADE`

Des index accélèrent les recherches les plus fréquentes : par date de départ et par couple de villes.



Structure de la table `trajets` : colonnes, clés étrangères et index.

8.4 Script de création

Le script complet est dans le fichier `database/schema.sql` du dépôt. Voici l'essentiel, la table `trajets`, qui est la plus riche en règles :

```
CREATE TABLE trajets (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    ville_depart_id     INT      NOT NULL,
    ville_arrivee_id    INT      NOT NULL,
    date_depart         DATETIME NOT NULL,
```

```
    date_arrivee        DATETIME NOT NULL,
    places_total        INT      NOT NULL,
    places_dispo        INT      NOT NULL,
    user_id             INT      NOT NULL,

    FOREIGN KEY (ville_depart_id)  REFERENCES villes(id),
    FOREIGN KEY (ville_arrivee_id) REFERENCES villes(id),
    FOREIGN KEY (user_id)          REFERENCES users(id),

    CONSTRAINT chk_villes_differentes CHECK (ville_depart_id <>
ville_arrivee_id),
    CONSTRAINT chk_dates_coherentes   CHECK (date_arrivee > date_depart),
    CONSTRAINT chk_places_total       CHECK (places_total BETWEEN 1 AND 9),
    CONSTRAINT chk_places_dispo       CHECK (places_dispo >= 0 AND
places_dispo <= places_total),

    INDEX idx_trajets_date_depart (date_depart),
    INDEX idx_trajets_villes (ville_depart_id, ville_arrivee_id)
);
```

La table `reservations` suit le même principe :

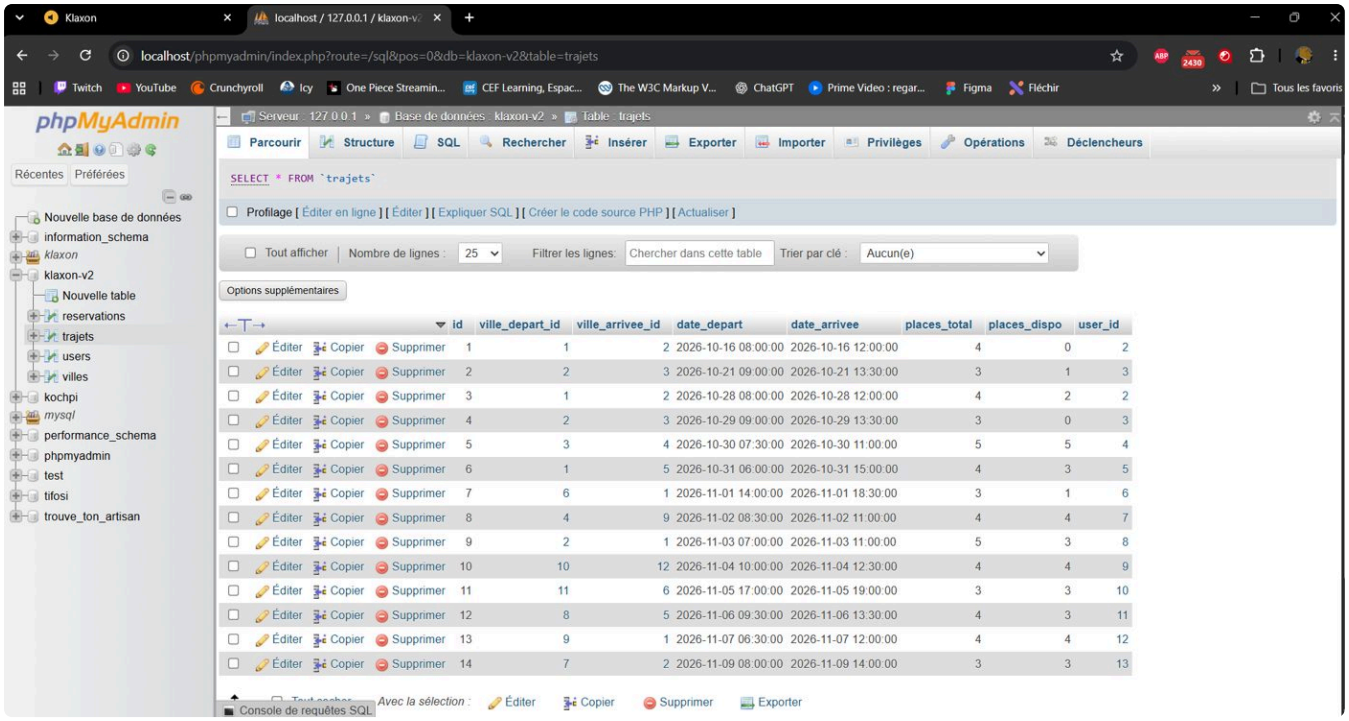
```
CREATE TABLE reservations (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    trajet_id   INT      NOT NULL,
    user_id     INT      NOT NULL,
    nb_places   INT      NOT NULL DEFAULT 1,
    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (trajet_id) REFERENCES trajets(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id)   REFERENCES users(id)   ON DELETE CASCADE,

    UNIQUE KEY uq_reservation_user_trajet (user_id, trajet_id),
    CONSTRAINT chk_nb_places CHECK (nb_places >= 1)
);
```

8.5 Jeu de données

Le fichier `database/seed.sql` remplit la base pour la démonstration : une soixantaine de villes françaises, des comptes de test dont les mots de passe sont déjà hachés, et des trajets d'exemple.



Quelques trajets du jeu de données.

9. Accès aux données SQL (compétence 5)

Le serveur ne mixe pas les requêtes SQL avec le reste du code : chaque table a son fichier dans le dossier `models`. Les autres parties du serveur appellent ces fonctions (`findById`, `create`, `remove`…) sans savoir comment la base est interrogée.

9.1 Connexion à MySQL

Le serveur utilise un *pool* de connexions : quelques connexions restent ouvertes et sont réutilisées, ce qui évite d'en ouvrir une nouvelle à chaque requête. Les identifiants viennent du fichier `.env`, jamais du code.

```
const pool = mysql.createPool({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  dateStrings: true,
});
```

9.2 Requêtes préparées

Toutes les requêtes utilisent des points d'interrogation (`?`) à la place des valeurs. MySQL reçoit d'abord la requête, puis les valeurs séparément : une valeur ne peut donc jamais être prise pour du code SQL. C'est la protection contre l'injection SQL (section 12).

```
async function findByEmail(email) {
  const [rows] = await pool.execute(
    `SELECT ${COLONNES}, password FROM users WHERE email = ?`, [email]
  );
  return rows[0];
}
```

Les colonnes sont listées une par une plutôt qu'avec `SELECT *` : le mot de passe haché n'est renvoyé que par cette requête, utilisée uniquement à la connexion.

9.3 Créer, lire, supprimer

Fonction	Ce qu'elle fait
`user.create`	Ajoute un compte (le rôle est toujours `user`)
`trajet.findAll`	Liste les trajets à venir, avec filtres facultatifs
`trajet.findById`	Un trajet avec le conducteur et ses coordonnées
`trajet.create`	Ajoute un trajet (au début, toutes les places sont libres)
`trajet.remove`	Supprime un trajet **de son conducteur**
`trajet.changerPlaces`	Retire ou rend une place
`reservation.create` / `remove`	Ajoute ou annule une réservation
`reservation.findByUser`	Les réservations d'un utilisateur

9.4 Requête avec filtres

Pour la recherche, la requête est complétée morceau par morceau selon les filtres choisis. Seul le texte de la requête change : les valeurs, elles, restent toujours des `?`.

```
let sql = SELECT_TRAJET + ' WHERE t.date_depart > NOW()';
const valeurs = [];

if (filtres.depart) {
  sql += ' AND t.ville_depart_id = ?';
  valeurs.push(filtres.depart);
}
if (filtres.date) {
  sql += ' AND DATE(t.date_depart) = ?';
  valeurs.push(filtres.date);
}

sql += ' ORDER BY t.date_depart';
const [rows] = await pool.execute(sql, valeurs);
```

9.5 Jointures

La liste des trajets a besoin du nom des villes et du conducteur, qui sont dans d'autres tables. Une jointure (`JOIN`) les rassemble en une seule requête. La table `villes` est jointe deux fois : une fois pour le départ, une fois pour l'arrivée.

```
SELECT t.id, t.date_depart, t.places_dispo,
       vd.nom AS ville_depart, va.nom AS ville_arrivee,
       u.prenom AS conducteur_prenom
FROM trajets t
JOIN villes vd ON vd.id = t.ville_depart_id
JOIN villes va ON va.id = t.ville_arrivee_id
JOIN users u ON u.id = t.user_id
```

9.6 Protéger les données des autres

Supprimer un trajet ou une réservation n'est possible que pour son propriétaire. La condition est dans la requête elle-même : si le trajet appartient à quelqu'un d'autre, rien n'est supprimé et le serveur répond « introuvable ».

```
async function remove(id, userId) {
  const [result] = await pool.execute(
    'DELETE FROM trajets WHERE id = ? AND user_id = ?', [id, userId]
  );
  return result.affectedRows > 0;
}
```

10. Accès aux données NoSQL (compétence 5)

Les avis sont stockés dans MongoDB, une base NoSQL. Au lieu de tables et de lignes, elle range des documents, un peu comme des fiches, regroupés dans une collection. Un avis est une fiche indépendante, avec un commentaire de longueur libre : MongoDB convient bien à ce genre de contenu.

10.1 Structure de la collection `avis`

Chaque document contient l'avis, et aussi le prénom de l'auteur, pour l'afficher sans devoir interroger MySQL.

```
{
  trajetId: 8,             // le trajet noté (numéro MySQL)
  conducteurId: 7,         // le conducteur noté
  auteurId: 2,             // le passager qui donne l'avis
  auteurPrenom: 'Sophie',
  note: 5,                 // entier de 1 à 5
  commentaire: 'Conducteur ponctuel',
  createdAt: Date
}
```

Image — Documents de la collection avis dans MongoDB Compass

Trois avis de la collection `avis`, vus dans MongoDB Compass.

Les numéros (`trajetId` , `conducteurId` , `auteurId`) sont ceux des tables MySQL. Les deux bases sont donc liées par ces numéros, mais MongoDB ne vérifie rien : c'est le serveur qui contrôle que le trajet existe et que l'auteur l'a réservé (section 11).

10.2 Connexion et index

Au démarrage, le serveur se connecte à MongoDB Atlas et crée deux index :

```
await client.connect();

// Un passager ne peut donner qu'un seul avis par trajet
await avis.createIndex({ trajetId: 1, auteurId: 1 }, { unique: true });
// Retrouver vite les avis d'un conducteur
await avis.createIndex({ conducteurId: 1 });
```

Le premier index est **unique** : même si le code serveur avait un oubli, la base refuserait un deuxième avis du même passager sur le même trajet. Le second accélère l'affichage des avis d'un conducteur.

Image — Index de la collection avis dans MongoDB Compass

Index de la collection : `_id` (automatique), l'index unique `trajetId` + `auteurId`, et `conducteurId`.

10.3 Ajouter un avis

```
async function create(unAvis) {
  await avis().insertOne(unAvis);
}
```

Si l'index unique refuse l'avis, MongoDB renvoie l'erreur numéro 11000. Le serveur la transforme en message clair (« Vous avez déjà donné votre avis sur ce trajet »).

10.4 Lire les avis d'un conducteur

La lecture renvoie les 50 avis les plus récents. On choisit les champs à renvoyer : l'identifiant interne de MongoDB, par exemple, ne sert pas à l'écran.

```
async function findByConducteur(conducteurId) {
  return avis()
    .find({ conducteurId }, { projection: { _id: 0, auteurPrenom: 1, note: 1,
commentaire: 1, createdAt: 1 } })
    .sort({ createdAt: -1 })
    .limit(50)
    .toArray();
}
```

10.5 Calculer la note moyenne

La moyenne est calculée par MongoDB lui-même, avec une *agrégation* : une série d'étapes qui traite les documents. Ici, on garde les avis du conducteur, puis on les regroupe pour calculer la moyenne des notes et leur nombre. Le serveur arrondit ensuite à une décimale.

```
async function statsConducteur(conducteurId) {
  const resultat = await avis()
    .aggregate([
      { $match: { conducteurId } },
      { $group: { _id: null, moyenne: { $avg: '$note' }, nombre: { $sum: 1 }
} },
    ])
    .toArray();
  return resultat[0]; // aucun résultat s'il n'y a pas d'avis
}
```

Étape	Rôle
`$match`	Garde seulement les avis de ce conducteur
`$group`	Regroupe tout en un résultat : `$avg` calcule la moyenne, `$sum: 1` compte les avis

Plutôt que de charger tous les avis dans le serveur pour les additionner, on laisse la base faire le calcul : c'est plus rapide et le code est plus court.

11. Composants métier côté serveur (compétence 6)

Les composants métier appliquent les règles de l'application. Ils sont dans le dossier `controllers` : chaque fonction reçoit une requête, vérifie qu'elle est correcte et autorisée, puis appelle les fonctions d'accès aux données des sections 9 et 10.

11.1 Organisation du serveur

Dossier	Rôle
`routes`	Associe chaque adresse (par exemple `POST /api/avis`) à une fonction, et dit si la connexion est obligatoire
`middlewares`	Vérifie le cookie de connexion avant d'exécuter la fonction
`controllers`	Applique les règles métier
`models`	Parle aux bases de données

Les routes de réservation, par exemple, sont toutes protégées par une seule ligne :

```
router.use(requireAuth); // toutes les routes ci-dessous demandent d'être
connecté
router.get('/', controller.mine);
router.post('/', controller.create);
router.delete('/:trajetId', controller.remove);
```

11.2 Réserver une place

Avant d'accepter une réservation, le serveur vérifie cinq choses, dans cet ordre. Dès qu'une règle n'est pas respectée, il s'arrête et renvoie un message précis.

Vérification	Réponse en cas d'échec
Le trajet existe	404 « Trajet introuvable »
Ce n'est pas son propre trajet	400
Le trajet n'est pas déjà parti	400 « Ce trajet est déjà parti »
Il reste au moins une place	409 « Ce trajet est complet »
Le passager n'a pas déjà réservé	409 « Vous avez déjà réservé ce trajet »

Si tout est correct, le serveur enregistre la réservation puis retire une place au trajet.

```
if (trajet.user_id === req.user.id) {
  return res.status(400).json({ message: 'Vous ne pouvez pas réserver votre
propre trajet' });
}
if (trajet.places_dispo < 1) {
  return res.status(409).json({ message: 'Ce trajet est complet' });
}
// ...
await Reservation.create(trajetId, req.user.id);
await Trajet.changerPlaces(trajetId, -1);
res.status(201).json({ message: 'Réservation confirmée' });
```

L'annulation fait l'inverse : elle supprime la réservation de l'utilisateur connecté, puis rend la place au trajet.

Limite connue : les deux écritures (réservation, puis nombre de places) ne sont pas regroupées dans une transaction. Si deux personnes réservaient la dernière place au même instant, les deux pourraient être acceptées. Les règles de la base (`places_dispo >= 0`) empêchent toutefois un nombre de places négatif. Je l'ai noté comme piste d'amélioration en conclusion.

11.3 Proposer et supprimer un trajet

Pour proposer un trajet, le serveur ne fait pas confiance au formulaire : il contrôle chaque champ, même si la page l'a déjà fait.

```
if (ville_depart_id === ville_arrivee_id) { /* 400 : départ et arrivée
différents */ }
if (new Date(date_depart) <= new Date()) { /* 400 : le départ doit être dans
le futur */ }
if (new Date(date_arrivee) <= new Date(date_depart)) { /* 400 : arrivée après
le départ */ }
if (!Number.isInteger(places_total) || places_total < 1 || places_total > 9)
{ /* 400 */ }

// Le conducteur est l'utilisateur connecté, jamais une valeur envoyée par le
formulaire
await Trajet.create(..., places_total, req.user.id);
```

Le conducteur est toujours l'utilisateur connecté : impossible de créer un trajet au nom de quelqu'un d'autre. De même, pour supprimer, un trajet inexistant et un trajet appartenant à un autre reçoivent la même réponse « introuvable », pour ne rien révéler.

11.4 Donner un avis et calculer la moyenne

C'est la fonctionnalité la plus complète : elle croise les deux bases de données. Les vérifications sont les suivantes.

Étape	Base utilisée	Si ça échoue
La note est un entier de 1 à 5, le commentaire fait 500 caractères au plus	Aucune	400
Le trajet existe	MySQL	404
L'auteur a réservé ce trajet	MySQL	403 « Seuls les passagers de ce trajet peuvent donner un avis »
Il n'a pas déjà donné d'avis sur ce trajet	MongoDB (index unique)	409
Enregistrement de l'avis	MongoDB	201

```
// Seuls les passagers ayant réservé ce trajet peuvent donner un avis
if (!(await Reservation.exists(trajet_id, req.user.id))) {
  return res.status(403).json({ message: 'Seuls les passagers de ce trajet
peuvent donner un avis' });
}

await Avis.create({
  trajetId: trajet_id,
  conducteurId: trajet.user_id, // pris dans la base, jamais dans la requête
  auteurId: req.user.id,        // l'utilisateur connecté
  auteurPrenom: req.user.prenom,
  note,
  commentaire: commentaire ? commentaire.trim() : '',
  createdAt: new Date(),
});
```

L'auteur de l'avis et le conducteur noté ne viennent jamais du formulaire : ils sont déduits de la connexion et de la base. Un utilisateur ne peut donc pas noter à la place d'un autre.

La note moyenne est calculée à la lecture (section 10.5), puis arrondie à une décimale. Un conducteur sans avis reçoit `null`, et l'écran affiche « Pas encore d'avis ».

```
res.json({
  conducteurId: id,
  moyenne: stats ? Math.round(stats.moyenne * 10) / 10 : null,
  nombre: stats ? stats.nombre : 0,
  avis,
});
```

12. Sécurité de l'application

La sécurité n'est pas ajoutée à la fin : elle est présente à chaque couche. Cette section regroupe les protections, en disant à chaque fois de quelle attaque elles protègent. Leur efficacité est testée en sections 13 et 14.

Risque	Protection	Où
Vol de mot de passe en base	Hachage bcrypt	Serveur
Vol de la session	Cookie `httpOnly` et `SameSite`	Serveur
Accès sans être connecté	Contrôle du jeton à chaque requête	Serveur
Injection SQL	Requêtes préparées	Serveur

Risque	Protection	Où
Injection NoSQL	Vérification des types	Serveur
XSS (code malveillant dans une page)	Échappement automatique de React, en-têtes `helmet`	Client et serveur
Force brute sur la connexion	Limitation à 5 échecs	Serveur
Accès aux données des autres (IDOR)	Condition sur le propriétaire	Serveur et base
Se donner des droits	Rôle imposé par le serveur	Serveur
Appels depuis un autre site	CORS et `SameSite`	Serveur
Fuite d'informations	Messages d'erreur génériques	Serveur
Secrets dans le code	Fichier `.env` hors du dépôt	Serveur

12.1 Mots de passe

Le mot de passe n'est jamais enregistré en clair : on stocke seulement son empreinte, calculée avec bcrypt. Cette fonction est volontairement lente et ajoute un « sel » aléatoire, ce qui rend les attaques très coûteuses. Un mot de passe de 8 caractères minimum est exigé.

```
const hash = await bcrypt.hash(password, 10);
```

12.2 Connexion et session

Après une connexion réussie, le serveur crée un jeton (JWT) qui contient seulement le numéro de l'utilisateur et expire au bout de 2 heures. Ce jeton est placé dans un cookie avec trois protections :

```
const cookieOptions = {
  httpOnly: true,       // le JavaScript de la page ne peut pas le lire
  sameSite: 'strict',   // jamais envoyé depuis un autre site
  secure: process.env.NODE_ENV === 'production', // HTTPS obligatoire en
production
  maxAge: 2 * 60 * 60 * 1000, // 2 heures
};
```

Image — Cookie de session dans les outils de développement de Chrome

Le cookie `klaxon_token` dans Chrome : les colonnes HttpOnly (cochée) et SameSite (Strict) confirment les protections.

- **`httpOnly`** : même si un pirate arrivait à exécuter du code dans la page, il ne pourrait pas voler le jeton.
- **`SameSite: strict`** : un site malveillant ne peut pas déclencher une action au nom de l'utilisateur connecté.
- **`secure`** : en production, le cookie ne circule qu'en HTTPS.

À chaque requête vers une route protégée, `requireAuth` vérifie le jeton, puis relit l'utilisateur en base : un compte supprimé perd son accès immédiatement.

```
const payload = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(payload.id);
if (!user) {
  return res.status(401).json({ message: 'Session invalide' });
}
req.user = user;
```

Le message d'erreur de connexion est le même que l'email n'existe pas ou que le mot de passe soit faux (« Email ou mot de passe incorrect »), pour ne pas aider un pirate à deviner quels emails ont un compte.

12.3 Limiter les tentatives de connexion

Pour bloquer les essais de mots de passe en série (force brute), la connexion est limitée à 5 échecs toutes les 15 minutes par adresse IP. Au-delà, le serveur répond « Trop de tentatives » (code 429).

```
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 5,
  skipSuccessfulRequests: true,
});
router.post('/login', loginLimiter, controller.login);
```

12.4 En-têtes HTTP de sécurité

La bibliothèque `helmet` ajoute à chaque réponse des en-têtes qui demandent au navigateur d'activer ses protections : n'exécuter que les scripts du site (Content-Security-Policy), toujours utiliser HTTPS (Strict-Transport-Security), ne pas deviner le type des fichiers (X-Content-Type-Options), refuser l'affichage du site dans le cadre d'un autre (X-Frame-Options). L'en-tête qui révèle la technologie du serveur (`X-Powered-By`) est supprimé. Le CORS n'autorise que l'adresse du site (`CLIENT_URL`).

```
app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
```

Image — En-têtes de sécurité renvoyés par l'API, vus dans Postman

Réponse de l'API dans Postman : les en-têtes ajoutés par helmet.

12.5 Injections (SQL, NoSQL)

Communes à toutes les applications, les injections consistent à envoyer du code déguisé en données. Deux protections :

- **SQL** : toutes les requêtes sont préparées (section 9.2). Une valeur n'est jamais exécutée comme du SQL.
- **NoSQL** : MongoDB accepte des objets comme `{ "$ne": null }` dans une requête. Le serveur vérifie donc le type de chaque valeur avant de l'utiliser : un numéro doit être un nombre entier, un texte doit être un texte.

```
if (typeof email !== 'string' || typeof password !== 'string') {
  return res.status(400).json({ message: 'Email et mot de passe obligatoires'
});
}
```

12.6 Faille XSS

Une faille XSS permet à un pirate d'afficher du code JavaScript dans la page d'un autre utilisateur, par exemple via un commentaire d'avis. React protège par défaut : tout texte affiché est traité comme du texte, jamais comme du code. Le projet n'utilise nulle part `dangerouslySetInnerHTML`, qui désactiverait cette protection. La politique de sécurité du contenu (`helmet`) est une deuxième barrière.

12.7 Droits d'accès

- **Contrôle du propriétaire** : on ne peut supprimer que son propre trajet ou sa propre réservation, grâce à la condition `AND user_id = ?` dans la requête (section 9.6). Cela empêche de supprimer les données d'un autre en changeant un numéro dans l'adresse.
- **Rôle imposé** : un nouveau compte a toujours le rôle `user`. Le serveur ne lit jamais le rôle dans le formulaire d'inscription, donc impossible de s'inscrire en administrateur.
- **Identité déduite de la connexion** : le conducteur d'un trajet et l'auteur d'un avis viennent toujours du jeton, jamais du formulaire.

12.8 Secrets et erreurs

Les secrets (mot de passe de la base, adresse MongoDB, clé de signature des jetons) sont dans un fichier `.env` qui n'est pas envoyé sur GitHub. Un fichier `.env.example` montre les noms attendus, sans les valeurs. Enfin, en cas d'erreur inattendue, le serveur renvoie seulement « Erreur du serveur » : le détail technique reste dans ses journaux.

```
res.status(status).json({ message: status === 500 ? 'Erreur du serveur' :
'Requête invalide' });
```

13. Jeu d'essai : donner un avis et calculer la note moyenne

J'ai choisi cette fonctionnalité car c'est la plus complète : elle utilise les deux bases de données, la connexion, les règles métier et l'affichage. Le principe d'un jeu d'essai : pour chaque action, on note le résultat **attendu** avant de la faire, puis le résultat **obtenu**, et on compare.

13.1 Conditions du test

- **Trajet testé** : n°8, Toulouse → Bordeaux, conducteur Chloé Roux, 4 places.
- **État de départ** : aucune réservation sur ce trajet, collection `avis` vide.
- **Comptes** : trois passagers (Sophie, Julien, Emma) et un utilisateur qui n'a pas réservé (Thomas).
- **Outils** : l'application dans Chrome, Postman pour envoyer des requêtes directes à l'API, MongoDB Compass pour contrôler la base.

13.2 Tableau des essais

N°	Entrée (action)	Résultat attendu	Résultat obtenu	Conforme
01	Sophie ouvre le trajet 8	4 places restantes, « Pas encore d'avis », pas de formulaire d'avis	Identique	Oui
02	Sophie clique sur « Réserver une place »	Confirmation, 3 places restantes, formulaire d'avis visible	Identique	Oui
03	Sophie envoie la note 5 et le commentaire « Conducteur ponctuel »	« 5 sur 5 · 1 avis », l'avis apparaît dans la liste	Identique	Oui

N°	Entrée (action)	Résultat attendu	Résultat obtenu	Conforme
04	Sophie renvoie un avis sur le même trajet	Refus : « Vous avez déjà donné votre avis sur ce trajet »	Identique	Oui
05	Contrôle dans Compass	1 document : trajet 8, conducteur 7, auteur 2, note 5	Identique	Oui
06	Julien réserve, puis envoie la note 2	Moyenne (5+2)/2 = 3,5 : « 3,5 sur 5 · 2 avis »	Identique	Oui
07	Emma réserve, puis envoie la note 4	Moyenne (5+2+4)/3 = 3,67 arrondie à 3,7 : « 3,7 sur 5 · 3 avis » ; 1 place restante	Identique	Oui
08	Thomas (non passager) envoie un avis par Postman	Refus 403 : « Seuls les passagers de ce trajet peuvent donner un avis »	Identique	Oui
09	Sophie envoie la note 8 par Postman	Refus 400 : « La note doit être un nombre entier entre 1 et 5 »	Identique	Oui
10	Sophie envoie la note `{"$gt": 0}` (tentative d'injection NoSQL)	Refus 400, même message	Identique	Oui
11	Consultation publique des avis du conducteur, sans connexion	Réponse 200, moyenne 3,7, 3 avis, sans les numéros d'auteur et de conducteur	Identique	Oui
12	Contrôle dans Compass : documents et index	3 documents ; index unique `trajetId` + `auteurId`	Identique	Oui

13.3 Captures des essais

Image — Essai 02 : réservation confirmée et formulaire d'avis

Essai 02 : après la réservation, la confirmation s'affiche et le formulaire d'avis apparaît.

Image — Essais 03 et 04 : premier avis, puis refus du doublon

Essais 03 et 04 : l'avis est enregistré, un deuxième avis du même passager est refusé.

Image — Essai 07 : note moyenne de 3,7 avec trois avis

Essai 07 : la note moyenne affichée est 3,7 sur 5, calculée sur 3 avis.

Image — Essai 08 : refus d'un utilisateur qui n'est pas passager

Essai 08 : un utilisateur qui n'a pas réservé reçoit une erreur 403.

Image — Essai 10 : tentative d'injection NoSQL refusée

Essai 10 : la note `{"$gt": 0}` est refusée avant d'atteindre MongoDB.

Les captures des essais 01, 05, 06, 09, 11 et 12 sont dans le dossier du projet ; les contrôles dans MongoDB Compass (essais 05 et 12) sont ceux des figures de la section 10.

13.4 Analyse des écarts

Les 12 essais donnent le résultat attendu : **aucun écart** entre l'attendu et l'obtenu.

- **Règles métier** : seul un passager du trajet peut noter (essai 08), et une seule fois (essai 04).
- **Données invalides** : une note hors de 1 à 5 (essai 09) et une tentative d'injection (essai 10) sont refusées par le serveur avant d'atteindre la base.
- **Calcul** : la moyenne (5+2+4)/3 = 3,67 est correctement arrondie à 3,7 (essais 07 et 11).
- **Double protection** : l'unicité d'un avis par passager est garantie aussi par l'index de MongoDB (essai 12), pas seulement par le code.

Une limite n'a pas été testée : deux réservations simultanées de la dernière place (section 11.2).

14. Veille sur les vulnérabilités de sécurité

Une veille de sécurité consiste à chercher les failles possibles et à les corriger. Je l'ai faite de deux façons : en simulant moi-même des attaques sur l'application, et en contrôlant les bibliothèques utilisées avec l'outil `npm audit`, qui les compare à une liste publique de failles connues. Chaque test simule une attaque ; il est réussi quand l'attaque est **bloquée**.

14.1 Attaques simulées

N°	Attaque simulée	Résultat attendu	Résultat obtenu	Bloquée
V-01	Voir le détail d'un trajet et les contacts du conducteur sans être connecté	Refus 401	401 « Vous devez être connecté », aucun contact renvoyé	Oui
V-02	Thomas supprime le trajet 8, qui appartient à Chloé	Refus 404	404 « Trajet introuvable »	Oui
V-03	Thomas annule une réservation qui n'est pas la sienne	Refus 404	404 « Réservation introuvable »	Oui
V-04	Vérifier que le trajet 8 existe toujours après ces attaques	Trajet intact	Trajet et places inchangés	Oui
V-05	Injection SQL à la connexion (`' OR 1=1 --`)	Refus	401 « Email ou mot de passe incorrect »	Oui
V-06	Injection SQL dans la recherche de trajets	Pas d'erreur SQL	400 « Filtre invalide » : le texte est refusé	Oui
V-07	Injection NoSQL à la connexion (`{"$ne":""}`)	Refus	400 : le mot de passe doit être du texte	Oui
V-08	Plusieurs mots de passe faux de suite (force brute)	Blocage après 5 échecs	429 « Trop de tentatives, réessayez dans 15 minutes »	Oui
V-09	Avis contenant `<script>alert(1)</script>`	Texte affiché tel quel, aucun script	Texte visible en clair, aucune alerte	Oui
V-10	Inscription avec `"role":"admin"` dans la requête	Compte créé avec le rôle `user`	Le serveur renvoie `"role": "user"`	Oui
V-11	`npm audit` : failles connues dans les bibliothèques	Aucune faille	**Failles trouvées**	Non, corrigé ensuite

Sur 11 tests, 10 attaques sont bloquées dès le départ. Les protections correspondantes sont décrites en section 12.

Image — Essai V-02 : suppression du trajet d'un autre utilisateur refusée

V-02 : Thomas essaie de supprimer le trajet de Chloé et reçoit une erreur 404.

Image — Essai V-08 : blocage après plusieurs tentatives de connexion

V-08 : après 5 échecs, le serveur répond 429 « Trop de tentatives ».

Image — Essai V-09 : le script saisi dans un avis est affiché comme du texte

V-09 : le code `<script>` d'un avis s'affiche comme du texte et ne s'exécute pas.

14.2 Faille trouvée et corrigée

L'audit des bibliothèques a trouvé trois failles réelles, dans des bibliothèques que je n'ai pas écrites mais que mon projet utilise :

Bibliothèque	Où	Gravité	Risque
`body-parser` 2.0.0 à 2.3.0 (utilisée par Express)	Serveur	Haute	Un message compressé mal formé peut bloquer le serveur (déni de service)
`ip-address` 10.7.3 ou moins (utilisée par la limitation des connexions)	Serveur	Moyenne	Une très longue chaîne peut bloquer le processus
`zod` avant 4.6.6 (outil de développement indirect)	Client	Haute	Épuisement de la mémoire

Image — Audit du serveur avant correction : 2 failles

Avant : `npm audit` signale 2 failles côté serveur (1 haute, 1 moyenne).

Correction. La commande `npm audit fix` met à jour les bibliothèques concernées vers une version corrigée. Je l'ai lancée côté serveur (3 paquets mis à jour) et côté client (1 paquet), puis j'ai relancé l'audit : **0 vulnérabilité** des deux côtés. La correction a été enregistrée dans une branche `fix/audit` puis fusionnée dans `main`.

Image — Audit du serveur après correction : 0 faille

Après : `found 0 vulnerabilities`.

14.3 Bilan et suite

La veille a confirmé que les protections de la section 12 fonctionnent, et elle a révélé une vraie faille, venue des dépendances. Ce type de faille apparaît avec le temps, même quand on n'a pas touché au code : c'est pourquoi `npm audit` doit être relancé régulièrement, par exemple avant chaque mise en ligne. Les failles de l'application PHP d'origine (mots de passe, injections, droits) ont toutes été traitées dans la version 2.

15. Conclusion et bilan

15.1 Ce qui a été réalisé

Klaxon v2 est une application de covoiturage complète : on peut créer un compte, chercher et proposer un trajet, réserver une place, puis noter le conducteur. Elle fonctionne sur ordinateur et sur téléphone, s'appuie sur deux bases de données adaptées à leur usage, et a été testée et attaquée pour vérifier sa sécurité. Le code est versionné sur GitHub, avec une branche par fonctionnalité.

15.2 Difficultés rencontrées

- **Reprendre une base PHP.** Les mots de passe du jeu de données d'origine avaient été hachés par PHP. Le serveur Node ne les reconnaissait pas tels quels : j'ai dû adapter leur préfixe au moment de la comparaison (section 12).
- **Faire parler deux bases.** MongoDB et MySQL ne se vérifient pas entre elles. C'est donc le serveur qui contrôle qu'un avis vient bien d'un passager (section 11.4).
- **Trouver une faille dans mes propres dépendances.** L'audit a révélé des failles dans des bibliothèques que je n'avais pas écrites, corrigées par une mise à jour (section 14).

15.3 Limites connues

Limite	Conséquence	Piste d'amélioration
Pas de transaction SQL pour la dernière place	Deux réservations simultanées de la dernière place pourraient être acceptées	Regrouper la réservation et la mise à jour des places dans une transaction

Limite	Conséquence	Piste d'amélioration
Un seul rôle d'utilisateur	Pas de modération	Utiliser le champ `role`, déjà présent en base, pour un rôle administrateur
Pas de modification d'un trajet	Il faut le supprimer et le recréer	Ajouter une page de modification
Menu mobile en haut, au lieu de la barre fixe en bas des maquettes	Moins pratique à une main	Créer une barre de navigation dédiée au téléphone
Application hébergée en local	Non accessible depuis internet	Mettre en ligne le client, l'API et les bases, avec HTTPS

15.4 Bilan personnel

Ce projet m'a permis de relier toutes les compétences de la formation dans une même application : de la maquette à la base de données, de l'interface React à la sécurité de l'API. Ce que j'en retiens surtout, c'est que la sécurité se pense dès le début, à chaque couche, et qu'on ne peut pas la juger sans la tester.

16. Annexes : la fonctionnalité « avis et note moyenne »

Ces annexes contiennent le code complet de la fonctionnalité présentée dans le jeu d'essai (section 13), d'abord côté navigateur (front-end), puis côté serveur (back-end).

Annexe A : front-end

A.1 `client/src/components/AvisForm.jsx` : le formulaire d'avis

```
import { useState } from 'react'
import { api } from '../api.js'

// Formulaire pour donner un avis (réservé aux passagers du trajet)
export default function AvisForm({ trajetId, onDone }) {
  const [note, setNote] = useState(5)
  const [commentaire, setCommentaire] = useState('')
  const [erreur, setErreur] = useState('')

  async function handleSubmit(event) {
    event.preventDefault()
    setErreur('')
```

```
    try {
      await api.post('/avis', { trajet_id: trajetId, note, commentaire })
      setCommentaire('')
      onDone() // la page recharge la liste des avis
    } catch (err) {
      setErreur(err.message) // ex : "Vous avez déjà donné votre avis sur ce
trajet"
    }
  }

  return (
    <form className="card avis-form" onSubmit={handleSubmit}>
      <h2>Donner votre avis</h2>

      <div className="note-buttons" role="group" aria-label="Note de 1 à 5">
        {[1, 2, 3, 4, 5].map((n) => (
          <button
            key={n}
            type="button"
            className={n === note ? 'note-btn actif' : 'note-btn'}
            aria-pressed={n === note}
            onClick={() => setNote(n)}
          >
            {n}
          </button>
        ))}
      </div>

      <label className="field">
        Commentaire (facultatif)
        <textarea rows="3" maxLength={500} value={commentaire} onChange={(e)
=> setCommentaire(e.target.value)} />
      </label>

      {erreur && (
        <p className="alert" role="alert">
          {erreur}
        </p>
      )}

      <button type="submit" className="btn btn-primary">
        Envoyer mon avis
      </button>
    </form>
  )
}
```

A.2 `client/src/pages/Detail.jsx` : la page du trajet (chargement, réservation, avis, moyenne)

```
import { useEffect, useState } from 'react'
import { Link, useParams } from 'react-router-dom'
import { api } from '../api.js'
import { useAuth } from '../auth/AuthContext.js'
import AvisForm from '../components/AvisForm.jsx'
import { formatDate, formatHeure } from '../utils/format.js'

export default function Detail() {
  const { id } = useParams() // le numéro du trajet, pris dans l'adresse
  const { user } = useAuth()

  const [trajet, setTrajet] = useState(null)
  const [avis, setAvis] = useState(null) // { moyenne, nombre, avis: [...] }
  const [aReserve, setAReserve] = useState(false)
  const [erreur, setErreur] = useState('')
  const [version, setVersion] = useState(0) // on l'augmente pour recharger
les données

  useEffect(() => {
    Promise.all([api.get('/trajets/' + id), api.get('/reservations')])
      .then(([dataTrajet, dataReservations]) => {
        setTrajet(dataTrajet.trajet)
        setAReserve(dataReservations.reservations.some((r) => r.trajet_id ===
dataTrajet.trajet.id))
        return api.get('/avis/conducteur/' + dataTrajet.trajet.user_id)
      })
      .then((dataAvis) => setAvis(dataAvis))
      .catch((err) => setErreur(err.message))
  }, [id, version])

  function recharger() {
    setVersion(version + 1)
  }

  async function handleReserver() {
    setErreur('')
    try {
      await api.post('/reservations', { trajet_id: trajet.id })
      recharger()
    } catch (err) {
      setErreur(err.message)
    }
  }
```

```
  if (erreur && !trajet) {
    return (
      <>
        <p className="alert" role="alert">
          {erreur}
        </p>
        <Link to="/">← Retour aux trajets</Link>
      </>
    )
  }
  if (!trajet || !avis) {
    return <p>Chargement…</p>
  }

  const monTrajet = trajet.user_id === user.id
  const complet = trajet.places_dispo === 0

  return (
    <>
      <p>
        <Link to="/">← Retour aux trajets</Link>
      </p>

      <div className="detail">
        <section className="detail-principal">
          <div className="card">
            <h1>
              {trajet.ville_depart} ({trajet.departement_depart}) →
{trajet.ville_arrivee} ({trajet.departement_arrivee})
            </h1>
            <div className="infos">
              <div>
                <span className="muted">Départ</span>
                <strong>
                  {formatDate(trajet.date_depart)} à
{formatHeure(trajet.date_depart)}
                </strong>
              </div>
              <div>
                <span className="muted">Arrivée</span>
                <strong>
                  {formatDate(trajet.date_arrivee)} à
{formatHeure(trajet.date_arrivee)}
                </strong>
              </div>
```

```
              <div>
                <span className="muted">Places</span>
                <strong>
                  {trajet.places_dispo} sur {trajet.places_total} disponibles
                </strong>
              </div>
            </div>
          </div>

          <div className="card">
            <h2>Votre conducteur</h2>
            <p className="conducteur-nom">
              {trajet.conducteur_prenom} {trajet.conducteur_nom}
            </p>
            <p className="muted">
              {avis.moyenne !== null ? (
                <>
                  <span className="etoile">★</span>
{avis.moyenne.toString().replace('.', ',')} sur 5 · {avis.nombre} avis
                </>
              ) : (
                "Pas encore d'avis"
              )}
            </p>
            <div className="infos">
              <div>
                <span className="muted">Téléphone</span>
                <strong>{trajet.conducteur_telephone}</strong>
              </div>
              <div>
                <span className="muted">Email</span>
                <strong>{trajet.conducteur_email}</strong>
              </div>
            </div>
          </div>

          <div className="card">
            <h2>Avis sur {trajet.conducteur_prenom}</h2>
            {avis.avis.length === 0 && <p className="muted">Aucun avis pour
le moment.</p>}
            {avis.avis.map((a, index) => (
              <div key={index} className="avis-item">
                <strong>{a.auteurPrenom}</strong> <span
className="etoile">★</span> {a.note} sur 5
                {a.commentaire && <p>{a.commentaire}</p>}
              </div>
```

```
            ))}
          </div>
        </section>

        <aside className="detail-cote">
          <div className="card">
            <p className="places-restantes">
              {complet ? 'Trajet complet' : `${trajet.places_dispo}
place${trajet.places_dispo > 1 ? 's' : ''} restante${trajet.places_dispo > 1
? 's' : ''}`}
            </p>

            {monTrajet && <p className="muted">C'est votre trajet : vous ne
pouvez pas le réserver.</p>}
            {aReserve && <p className="succes">Réservation confirmée : vous
avez une place sur ce trajet.</p>}
            {erreur && (
              <p className="alert" role="alert">
                {erreur}
              </p>
            )}

            {!monTrajet && !aReserve && (
              <button className="btn btn-accent btn-large btn-block" onClick=
{handleReserver} disabled={complet}>
                Réserver une place
              </button>
            )}
          </div>

          {aReserve && <AvisForm trajetId={trajet.id} onDone={recharger} />}
        </aside>
      </div>
    </>
  )
}
```

Annexe B : back-end

B.1 `server/src/routes/avis.routes.js` : les adresses de l'API

```
const express = require('express');
const controller = require('../controllers/avis.controller');
const { requireAuth } = require('../middlewares/auth.middleware');

const router = express.Router();

router.post('/', requireAuth, controller.create);
router.get('/conducteur/:id', controller.conducteur); // public

module.exports = router;
```

B.2 `server/src/middlewares/auth.middleware.js` : le contrôle de connexion

```
const jwt = require("jsonwebtoken");
const User = require("../models/user.model");

// Bloque la requête si l'utilisateur n'est pas connecté.
// Sinon, il met l'utilisateur dans req.user pour la suite.
async function requireAuth(req, res, next) {
  const token = req.cookies.klaxon_token;
  if (!token) {
    return res.status(401).json({ message: "Vous devez être connecté" });
  }

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(payload.id);
    if (!user) {
      return res.status(401).json({ message: "Session invalide" });
    }
    req.user = user;
    next();
  } catch (err) {
    // jeton falsifié ou expiré
    res.status(401).json({ message: "Session invalide ou expirée" });
  }
}

module.exports = { requireAuth };
```

B.3 `server/src/controllers/avis.controller.js` : les règles métier

```
const Avis = require('../models/avis.model');
const Trajet = require('../models/trajet.model');
const Reservation = require('../models/reservation.model');

// POST /api/avis  { "trajet_id": 3, "note": 4, "commentaire": "Très sympa" }
async function create(req, res) {
  const { trajet_id, note, commentaire } = req.body;

  if (!Number.isInteger(trajet_id)) {
    return res.status(400).json({ message: 'Trajet invalide' });
  }
  if (!Number.isInteger(note) || note < 1 || note > 5) {
    return res.status(400).json({ message: 'La note doit être un nombre
entier entre 1 et 5' });
  }
  if (commentaire !== undefined && (typeof commentaire !== 'string' ||
commentaire.length > 500)) {
    return res.status(400).json({ message: 'Commentaire invalide (500
caractères maximum)' });
  }

  const trajet = await Trajet.findById(trajet_id);
  if (!trajet) {
    return res.status(404).json({ message: 'Trajet introuvable' });
  }

  // Seuls les passagers ayant réservé ce trajet peuvent donner un avis
  if (!(await Reservation.exists(trajet_id, req.user.id))) {
    return res.status(403).json({ message: 'Seuls les passagers de ce trajet
peuvent donner un avis' });
  }

  try {
    await Avis.create({
      trajetId: trajet_id,
      conducteurId: trajet.user_id, // pris dans la base, jamais dans la
requête
      auteurId: req.user.id, // l'utilisateur connecté
      auteurPrenom: req.user.prenom,
      note,
      commentaire: commentaire ? commentaire.trim() : '',
      createdAt: new Date(),
    });
  } catch (err) {
```

```
    // 11000 = l'index unique de MongoDB refuse un deuxième avis du même
passager sur ce trajet
    if (err.code === 11000) {
      return res.status(409).json({ message: 'Vous avez déjà donné votre avis
sur ce trajet' });
    }
    throw err;
  }

  res.status(201).json({ message: 'Avis enregistré' });
}

// GET /api/avis/conducteur/:id  -> note moyenne + liste des avis d'un
conducteur
async function conducteur(req, res) {
  const id = Number(req.params.id);
  if (!Number.isInteger(id)) {
    return res.status(400).json({ message: 'Conducteur invalide' });
  }

  const stats = await Avis.statsConducteur(id);
  const avis = await Avis.findByConducteur(id);

  res.json({
    conducteurId: id,
    moyenne: stats ? Math.round(stats.moyenne * 10) / 10 : null, // arrondie
à 1 décimale
    nombre: stats ? stats.nombre : 0,
    avis,
  });
}

module.exports = { create, conducteur };
```

B.4 `server/src/models/avis.model.js` : l'accès à MongoDB

```
const { getMongoDb } = require("../config/mongo");

// Un avis ressemble à ça dans MongoDB :
// { trajetId: 3, conducteurId: 2, auteurId: 5, auteurPrenom: 'Sophie', note:
4, commentaire: '...', createdAt: Date }

function avis() {
  return getMongoDb().collection("avis");
}
```

```
async function create(unAvis) {
  await avis().insertOne(unAvis);
}

// Les 50 derniers avis reçus par un conducteur
async function findByConducteur(conducteurId) {
  return avis()
    .find(
      { conducteurId },
      {
        projection: {
          _id: 0,
          trajetId: 1,
          auteurPrenom: 1,
          note: 1,
          commentaire: 1,
          createdAt: 1,
        },
      },
    )
    .sort({ createdAt: -1 })
    .limit(50)
    .toArray();
}

// Note moyenne et nombre d'avis d'un conducteur (calculés par MongoDB)
async function statsConducteur(conducteurId) {
  const resultat = await avis()
    .aggregate([
      { $match: { conducteurId } },
      {
        $group: { _id: null, moyenne: { $avg: "$note" }, nombre: { $sum: 1 }
},
      },
    ])
    .toArray();
  return resultat[0]; // undefined s'il n'y a aucun avis
}

module.exports = { create, findByConducteur, statsConducteur };
```

B.5 `server/src/config/mongo.js` : connexion et index

```
const { MongoClient } = require('mongodb');

const client = new MongoClient(process.env.MONGODB_URI);

// Donne accès à la base MongoDB (à utiliser après connectMongo)
function getMongoDb() {
  return client.db(process.env.MONGODB_DB);
}

// Ouvre la connexion à MongoDB et prépare la collection des avis
async function connectMongo() {
  await client.connect();

  // Index unique : un passager ne peut donner qu'un seul avis par trajet
(garanti par la base)
  await getMongoDb().collection('avis').createIndex({ trajetId: 1, auteurId:
1 }, { unique: true });
  // Index pour retrouver vite les avis d'un conducteur
  await getMongoDb().collection('avis').createIndex({ conducteurId: 1 });
}

module.exports = { connectMongo, getMongoDb };
```

B.6 `server/src/models/reservation.model.js` : vérifier qu'un passager a réservé (extrait)

```
// Est-ce que cet utilisateur a déjà réservé ce trajet ?
async function exists(trajetId, userId) {
  const [rows] = await pool.execute(
    "SELECT id FROM reservations WHERE trajet_id = ? AND user_id = ?",
    [trajetId, userId],
  );
  return rows.length > 0;
}
```